



저작자표시-비영리-변경금지 2.0 대한민국

이용자는 아래의 조건을 따르는 경우에 한하여 자유롭게

- 이 저작물을 복제, 배포, 전송, 전시, 공연 및 방송할 수 있습니다.

다음과 같은 조건을 따라야 합니다:



저작자표시. 귀하는 원저작자를 표시하여야 합니다.



비영리. 귀하는 이 저작물을 영리 목적으로 이용할 수 없습니다.



변경금지. 귀하는 이 저작물을 개작, 변형 또는 가공할 수 없습니다.

- 귀하는, 이 저작물의 재이용이나 배포의 경우, 이 저작물에 적용된 이용허락조건을 명확하게 나타내어야 합니다.
- 저작권자로부터 별도의 허가를 받으면 이러한 조건들은 적용되지 않습니다.

저작권법에 따른 이용자의 권리는 위의 내용에 의하여 영향을 받지 않습니다.

이것은 [이용허락규약\(Legal Code\)](#)을 이해하기 쉽게 요약한 것입니다.

[Disclaimer](#)

Ph.D. Dissertation of Engineering

Investigating and Improving
DRAM-Based Memory System Security
by Analyzing and Exploiting
DRAM Operations

DRAM 동작 분석 및 활용을 통한
DRAM 기반 메모리 시스템 보안 분석 및 개선

August 2025

Graduate School of
Convergence Science and Technology
Seoul National University
Intelligence and Information Major

Minbok Wi

Investigating and Improving DRAM-Based Memory System Security by Analyzing and Exploiting DRAM Operations

Advisor Jung Ho Ahn

Submitting a Ph.D. Dissertation of
Engineering

August 2025

Graduate School of
Convergence Science and Technology
Seoul National University
Intelligence and Information Major

Minbok Wi

Confirming the Ph.D. Dissertation written by
Minbok Wi

August 2025

Chair	<u>Jangwoo Kim</u>	(Seal)
Vice Chair	<u>Jung Ho Ahn</u>	(Seal)
Examiner	<u>Nam Sung Kim</u>	(Seal)
Examiner	<u>Jungrae Kim</u>	(Seal)
Examiner	<u>Jinho Lee</u>	(Seal)

Abstract

Dynamic Random-Access Memory (DRAM) is the most commonly used main memory device in modern computer systems, directly determining overall system performance, reliability, and security. However, the increasing complexity of DRAM architectures, coupled with the limited visibility and control provided by the primary-secondary interface, hinders system-level analysis of performance, reliability, and security. Therefore, it is essential to have a precise understanding and analysis of the behavior of DRAM-based main memory systems to design high-performance, highly reliable, and secure computing systems.

This dissertation proposes novel techniques to improve the security and reliability of DRAM-based memory systems through a systematic analysis and utilization of DRAM internal operations. First, we propose a new DRAM architecture designed to mitigate the RowHammer vulnerability, a representative reliability and security threat in DRAM. This architecture leverages internal DRAM commands to facilitate efficient in-DRAM data copying. Next, we analyze the system-level memory access latency based on the behaviors of DRAM and memory controllers. We also develop microbenchmarks that infer DRAM internal operations. Building upon this analysis, we introduce a reverse-engineering technique to uncover the structural and operational characteristics of commercial DRAM and memory controllers.

First, we propose a new in-DRAM RowHammer protection solution named

Shuffling Aggressor DRAM Rows (SHADOW). SHADOW dynamically randomizes DRAM row mapping information, mitigating newly introduced non-adjacent RowHammer attacks and preventing an attacker from targeting a specific victim row that may hold critical data. SHADOW also utilizes the recently introduced JEDEC RFM interface to enable in-DRAM RowHammer mitigation without any DRAM interface changes. To realize the in-DRAM row-shuffle operation with low performance and energy overhead, we introduce novel DRAM microarchitecture optimization techniques. We demonstrate the strong probabilistic protection of SHADOW against RowHammer attacks through adversarial pattern analysis and highlight the compelling performance, area, and energy overheads compared to those of state-of-the-art hardware-based RowHammer prevention solutions.

Second, we present a timing-based analysis method based on an in-depth understanding of the memory system’s internal structure, behavior, and timing characteristics. Decomposing DRAM address mappings into component-level functions is critical for understanding memory behavior and enabling precise RowHammer attacks, yet existing reverse-engineering methods fall short. We introduce novel timing-based techniques leveraging DRAM refresh intervals and consecutive access latencies to infer component-specific functions. Based on this, we present Sudoku, the first software-based tool to automatically decompose full DRAM address mappings into channel, rank, bank group, and bank functions while identifying row and column bits. We validate Sudoku’s effectiveness by successfully decomposing mappings on recent Intel and AMD processors.

These proposed approaches, which utilize internal DRAM commands and

analyze system-level memory access latency based on DRAM’s internal structure and behavior, can enhance the performance, security, and reliability of modern DRAM-based memory systems. The proposed SHADOW demonstrates that processing-using-memory techniques, such as in-DRAM row-copy operations, can be utilized not only to improve system performance but also to enhance security and reliability. Additionally, our detailed analysis of DRAM and memory controller behavior, along with our latency-based reverse-engineering methodology, can provide a foundation for precise evaluations of performance, security, and reliability in increasingly complex DRAM-based memory systems. Moreover, the provided software tool and experimental results provide valuable references for future research and development in DRAM-based memory systems.

Keywords: Memory System, DRAM, Memory Controller, RowHammer, Timing Channel Analysis, Microbenchmarking, DRAM Address Mapping, Reverse-Engineering

Student Number: 2018-24285

Contents

Abstract	i
Contents	iv
List of Figures	vii
List of Tables	ix
Chapter 1 Introduction	1
1.1 Research Contributions	4
1.2 Outline	5
Chapter 2 Background	7
2.1 DRAM-Based Main Memory Systems	7
2.2 DRAM Operations and Timings	9
2.3 DRAM Read-Disturbance Errors	10
2.4 DRAM Address Mappings	13

Chapter 3 Preventing RowHammer in DRAM with Intra-Subarray

Row Shuffling	16
3.1 Overview	16
3.2 Threat Model	17
3.3 Related Work	18
3.3.1 RowHammer Mitigating Actions and Blast-Attacks	18
3.3.2 Implementation Locations and Counter Structures	21
3.4 SHADOW	22
3.4.1 Overview	22
3.4.2 High-Level Row-Shuffle Operations	23
3.4.3 Incremental Refresh	25
3.4.4 Implications of Row-Shuffle in DRAM	26
3.5 SHADOW Hardware Architecture	26
3.5.1 Remapping-Row	27
3.5.2 Subarray Pairing	28
3.5.3 SHADOW Controller	30
3.6 SHADOW Detailed Operation and Timing	30
3.6.1 Operation and Timing on ACT	31
3.6.2 Operation and Timing on RFM	31
3.7 Evaluation	36
3.7.1 Security Analysis	36
3.7.2 SPICE Circuit Simulation	40
3.7.3 Performance Overhead Analysis	42
3.7.4 Area and Power Analysis	50

3.8	Discussion	53
3.8.1	Hardware RNG Unit in DRAM	53
3.8.2	Soft Post Package Repair (sPPR)	54
3.8.3	Optimizing RFM Interface	55
3.8.4	Per-Row Activation Counting (PRAC)	55

Chapter 4 Decomposing DRAM Address Mapping into Component Functions 57

4.1	Overview	57
4.2	Related Work	58
4.2.1	Reverse-Engineering DRAM Address Mappings	58
4.2.2	Limitations of Prior Reverse-Engineering Methods	60
4.3	Understanding How Systems Configure Memory	61
4.4	Timing Channels for Component Function Identification	64
4.4.1	Refresh Intervals	64
4.4.2	Consecutive Memory Accesses	66
4.5	Sudoku	68
4.5.1	Generating Desired Memory Addresses	69
4.5.2	Validating the System of Hash Functions	70
4.5.3	Decomposing DRAM Address Mappings	71
4.6	Results	73
4.6.1	Results: Intel Core Processor	73
4.6.2	Results: AMD Zen 4 Processor	74
4.7	Discussion	74

4.7.1	Exploiting Memory Error Log	74
4.7.2	Extending Sudoku to Other DRAMs and Hash Functions	75
4.7.3	Use Cases of Sudoku	76
Chapter 5 Conclusion		79
5.1	Summary	79
5.2	Future Work	81
Bibliography		83
국문초록		114

List of Figures

Figure 2.1	An organization of typical DRAM-based main memory systems	8
Figure 2.2	Three steps of typical RowHammer attacks	12
Figure 2.3	Overview of DRAM address mappings	13
Figure 3.1	Hardware-based RowHammer mitigating actions	19
Figure 3.2	Operational flows of SHADOW	22
Figure 3.3	Operations of SHADOW on RFM commands	25
Figure 3.4	Hardware architecture of SHADOW	27
Figure 3.5	Detailed operational flows of SHADOW on ACT commands	32
Figure 3.6	Detailed operational flows of SHADOW on RFM commands	33
Figure 3.7	SHADOW operation timing diagram	34
Figure 3.8	Relative performance of SHADOW on single-threaded workloads	44

Figure 3.9	Relative performance of SHADOW on multi-threaded workloads	45
Figure 3.10	Relative performance of SHADOW on multi-programmed workloads	46
Figure 3.11	Relative performance of SHADOW with various tRCD parameters	47
Figure 3.12	Relative performance of SHADOW with various blast radius	50
Figure 3.13	Relative performance of SHADOW with various H_{cnt} . .	51
Figure 3.14	Relative system-level power consumption of SHADOW .	53
Figure 4.1	Process of reading MCHBAR registers	63
Figure 4.2	Measured refresh-induced latency spikes	65
Figure 4.3	Theoretical timing differences between two consecutive memory reads	66
Figure 4.4	The Distribution of Consecutive Memory Access Latencies	67
Figure 4.5	An overview of Sudoku	68
Figure 4.6	Process of verifying injectivity and finding additional functions	70

List of Tables

Table 2.1	Terminologies and abbreviations for DRAM read-disturbance errors	11
Table 3.1	SHADOW-specific terminologies and abbreviations	24
Table 3.2	RowHammer-induced bitflip probability of SHADOW . .	39
Table 3.3	Timing values of SHADOW from the SPICE circuit simulation	40
Table 3.4	Actual machine system configurations for performance evaluation	42
Table 4.1	Comparison of capabilities of prior tools for reverse-engineering DRAM address mappings	61
Table 4.2	MCHBAR register information and MC timing parameters	63
Table 4.3	Results of component-level timing channel analyses	64
Table 4.4	Tested system configurations of Sudoku	73
Table 4.5	Results of Sudoku	78

Chapter 1

Introduction

Dynamic Random-Access Memory (DRAM) is becoming increasingly complex, introducing new security challenges in computer systems. DRAM is the most widely used memory device in modern computer systems due to its performance and cost efficiency. However, as DRAM technology continues to advance, DRAM deploys more complex internal architectures and adds various features

This chapter is based on the following published and submitted works: [149, 150].

- Minbok Wi, Jaehyun Park, Seoyoung Ko, Michael Jaemin Kim, Nam Sung Kim, Eojin Lee, and Jung Ho Ahn,
“*SHADOW: Preventing Row Hammer in DRAM with Intra-Subarray Row Shuffling*,”
In *Proceedings of the 29th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.
© 2023 IEEE. Reprinted with permission.
- Minbok Wi, Seungmin Baek, Seonyong Park, Mattan Erez, and Jung Ho Ahn,
“*Sudoku: Decomposing DRAM Address Mapping into Component Functions*,”
arXiv preprint arXiv:2506.15918, 2025. Presented at DRAMSec 2025 co-located with ISCA 2025.

and commands. This growing complexity hinders the analysis of DRAM and memory controller behaviors in DRAM-based memory systems.

The RowHammer phenomenon is a significant security issue in DRAM, where frequent activations on a specific row can cause unintended bitflips in adjacent rows. Since its public demonstration in 2014 [68], RowHammer has emerged as a critical threat to computer systems. Numerous experimental analyses have been conducted to study DRAM read-disturbance errors [18, 19, 36, 64, 84, 97, 99, 136, 160]. These studies have led to extensive research on various RowHammer attack techniques [4, 20, 23, 29, 32, 33, 49, 50, 57, 60, 70, 84, 112, 133, 135, 137, 157, 165] and the exploitations [3, 17, 74, 79, 161] resulting from RowHammer-induced bitflips. In response, a variety of defense solutions have been explored and proposed, including both hardware-based [7, 12, 56, 59, 62, 66–68, 77, 86, 98, 101, 106–109, 115, 117, 118, 120, 122, 123, 131, 132, 142, 152, 154, 155, 158, 163] and software-based methods [8, 10, 26, 72, 82, 119, 138, 156, 166], to protect against RowHammer attacks.

An in-depth understanding of internal architecture and operations of DRAM is essential for ensuring the security of DRAM-based memory systems. For example, new RowHammer attack techniques [13, 29, 60, 70, 151] and vulnerabilities within memory systems [22, 79, 87, 95, 104, 153] continue to emerge, often exploiting the latest DRAM features and our incomplete understanding of its behavior. Furthermore, how the memory controller manages DRAM operations directly affects the performance of the memory system and can lead to security vulnerabilities. This highlights the importance of analyzing how DRAM-based memory systems operate.

To address the increasing threat of DRAM read-disturbance errors, there is a growing demand to investigate and improve the security of DRAM-based memory systems [40, 48, 88, 110, 139, 144, 146, 149]. This dissertation begins by exploring methods to leverage DRAM operations to enhance the security of the memory systems. Next, we analyze DRAM operations and their behaviors to understand the interaction between DRAM and the memory controller, allowing us to model DRAM-based memory systems more accurately.

First, we propose SHADOW, a novel solution for mitigating RowHammer attacks within DRAM. This approach leverages in-DRAM row-copy operations that exploit the DRAM activation command. By breaking the spatial relationship between aggressor and victim rows, SHADOW provides a robust and efficient defense against non-adjacent RowHammer attacks. SHADOW also utilizes the recently introduced refresh management (RFM) interface and maintains intrinsic counter structures within RFM, thereby eliminating the need for tracking structures on the memory controller side and any modifications to the DDR interface.

Second, we introduce new timing channels in DRAM-based memory systems that utilize refresh intervals and consecutive memory accesses. By analyzing DRAM internal operations and timing behaviors of DRAM, we have developed Sudoku, a software-based reverse-engineering tool that decomposes DRAM addressing functions into their component-level functions. This tool enables a detailed analysis of memory controller operations in commercial processors, providing a deeper understanding of how DRAM and the memory controller interact in modern computer systems.

1.1 Research Contributions

The contribution of this dissertation is summarized as follows:

- We propose SHADOW, a new in-DRAM RowHammer prevention solution that leverages the JEDEC RFM interface to perform row-shuffle operations, dynamically breaks the adjacency of an aggressor row and the corresponding victim rows.
- SHADOW utilizes high-throughput intra-subarray row-copy operations, minimizing the performance penalty of row-shuffle operations.
- We implement row-shuffle inside DRAM subarrays by storing remapping information per subarray, eliminating the need for an additional SRAM or CAM structure. We propose various microarchitecture optimization techniques to minimize the timing overhead incurred when reading the remapping information.
- We also introduce an incremental refresh scheme, which operates simultaneously with row-shuffle, to enhance the protection capability against RowHammer attacks.
- We show that SHADOW has a negligible impact on both performance and power consumption by conducting experiments on actual computer systems and architectural simulations. SHADOW provides a strong defense against blast-attacks with the advantage of row-shuffle operations.
- We analyze timing channels present in modern DRAM-based memory

systems, focusing on observable variations of memory access latency from refresh intervals and consecutive memory accesses.

- Based on our timing channel analysis, we develop Sudoku, a software-based tool that discovers full DRAM address mappings and decomposes these mappings into DRAM internal component functions.
- We identify row and column bits precisely without requiring physical access to the system, and we verify the results by confirming the injectivity of the derived mapping. We find DRAM component functions in recent Intel and AMD processors with various memory configurations.
- We open-source our tool to support research on the reverse-engineering of DRAM address mappings and the analysis of DRAM-based memory systems.

1.2 Outline

The dissertation is organized as follows: Chapter 2 provides background on the DRAM-based main memory system. It describes the organization, operations, and timings of DRAM. This chapter also introduces the security issues related to DRAM, such as read-disturbance errors (*e.g.*, RowHammer), and details DRAM address mapping. Chapter 3 introduces SHADOW (Shuffling Aggressor DRAM Rows), a hardware-based RowHammer mitigation solution. This chapter presents the design, implementation, and evaluation of the proposed RowHammer mitigation solution. Chapter 4 provides the timing channel analysis of modern DRAM-based memory systems, and based on this analysis,

proposes Sudoku—a novel software-based reverse-engineering tool to recover full DRAM address mappings and decompose DRAM addressing functions into component-level functions. Finally, Chapter 5 concludes the dissertation and suggests directions for future research in memory system security and DRAM operation analysis.

Chapter 2

Background

2.1 DRAM-Based Main Memory Systems

A DRAM-based main memory system is hierarchically organized, as illustrated in Figure 2.1. At the top level, the system comprises multiple memory channels, each connected to one or more Dual In-Line Memory Modules (DIMMs),

This chapter is based on the following published and submitted works: [149, 150].

- Minbok Wi, Jaehyun Park, Seoyoung Ko, Michael Jaemin Kim, Nam Sung Kim, Eojin Lee, and Jung Ho Ahn,
“*SHADOW: Preventing Row Hammer in DRAM with Intra-Subarray Row Shuffling*,”
In *Proceedings of the 29th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.
© 2023 IEEE. Reprinted with permission.
- Minbok Wi, Seungmin Baek, Seonyong Park, Mattan Erez, and Jung Ho Ahn,
“*Sudoku: Decomposing DRAM Address Mapping into Component Functions*,”
arXiv preprint arXiv:2506.15918, 2025. Presented at DRAMSec 2025 co-located with ISCA 2025.

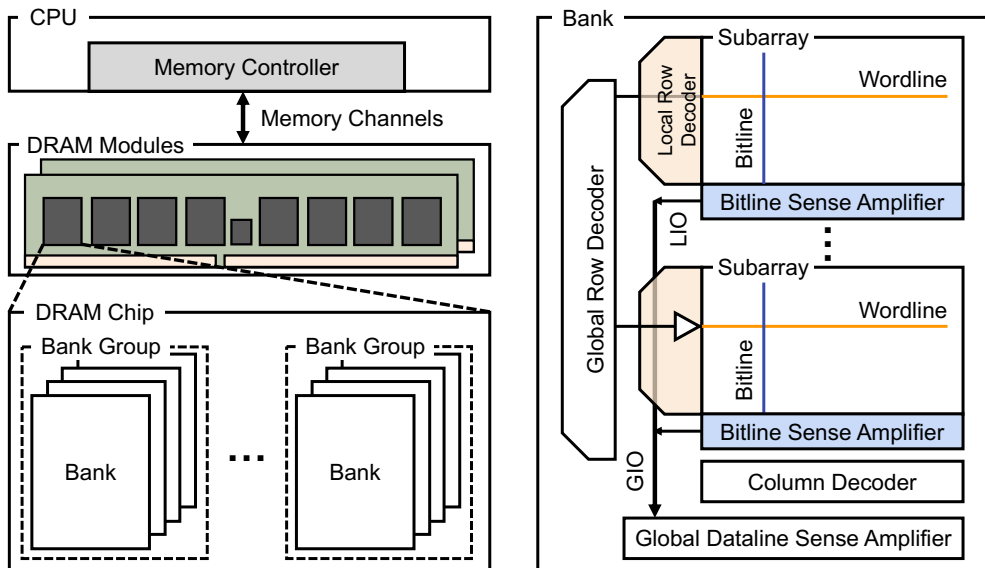


Figure 2.1: An organization of typical DRAM-based main memory systems.

connected to memory controllers (MCs).

Each DRAM module (DIMM) consists of multiple ranks, which are further divided into bank groups and banks. A DRAM bank comprises multiple subarrays, each organized as a two-dimensional array of rows and columns of DRAM cells. These DRAM cells are indexed by wordlines (rows) and bitlines (columns), respectively. Modern standards such as DDR5 [55] further increase complexity by introducing sub-channels and doubling the number of bank groups compared to DDR4 [51].

Each bitline is connected to a bitline sense amplifier (BSA), also known as a local row buffer, which temporarily stores row data. One BSA is connected to the global IO (GIO) via the local IO (LIO). To access (or activate) a row, the global row decoder first selects a subarray, and then the local row decoder

selects a specific row within the subarray.

2.2 DRAM Operations and Timings

Following the primary-secondary communication protocol of DDR (Double Data Rate) memory interfaces, a modern memory controller (MC) issues DRAM commands to a DRAM device in a synchronous manner, with JEDEC-specified fixed timings [51, 53–55]. Accessing data requires an MC to issue commands following specific timing constraints. An ACT (activate) command opens a target row within a bank. After a delay of t_{RCD} , RD (read), or WR (write) commands access specific columns in the open, activated row. Accessing a different row within the same bank necessitates a PRE (precharge) command and incurs an additional delay (t_{RP}) prior to the next activation. This sequence results in higher latency for same-bank accesses compared to row hits, creating the well-known row-buffer conflict timing channel.

Beyond row-buffer conflicts, other DRAM operations create observable timing variations [29]. DRAM cells require periodic refresh operations (REF) to retain data, which temporarily block memory accesses for t_{RFC} cycles and occur at an average interval of t_{REFI} . Modern DRAM devices support various types of refresh operations, such as all-bank, fine-grained, or same-bank refreshes, to reduce performance overhead from refresh operations [51, 54, 55]. MCs may implement those various refresh schemes, potentially affecting access latency differently depending on the target addresses.

DRAM employs mandatory timing parameters for consecutive accesses,

such as t_{RRD} (row-to-row delay) and t_{CCD} (column-to-column delay), which vary depending on whether the accesses target the same or different bank groups [51, 53–55]. However, rather than directly using these DRAM timing parameters, MCs use their own set of timing parameters—such as t_{RDRD} , t_{RDWR} , t_{WRRD} , and t_{WRWR} —based on the sequence and type of memory accesses [2, 25, 42, 43]. These timing parameters are more complex, as they depend not only on the bank group and bank, but also on the rank and DIMM. Moreover, they are non-SPD related timings whose values vary based on the system platform and configuration, and are typically determined during DDR training for optimal performance. While often small, these timing differences between consecutive accesses represent another potential channel for inferring address mapping details.

2.3 DRAM Read-Disturbance Errors

Modern DRAM devices are vulnerable to DRAM read-disturbance errors, such as RowHammer [68] and RowPress [84], which incur arbitrary bitflips in nearby DRAM rows. RowHammer is the phenomenon of repeatedly activating a specific (aggressor) row to cause bitflips of its physically adjacent (victim) rows. The minimum activation count to incur the RowHammer bitflips is referred to as the Hammer Count (H_{cnt}) [64]. RowPress, in contrast, causes bitflips based on keeping a row open for an extended period. Both the RowHammer and RowPress phenomena pose a critical threat to computer systems, as they introduce the risk of bitflips on any row, potentially leading to system integrity failures.

Table 2.1: Terminologies and abbreviations for DRAM read-disturbance errors

Symbol	Description
H_{cnt}	Hammer Count. The minimum activation count for incurring the bit-flip by hammering the row.
t_{REFI}	An average timing interval between two auto-refreshes.
$t_{\text{RFC}}, t_{\text{RFC}}_{\text{sb}}$	Refresh cycle for all bank refreshes and same bank refreshes, respectively.
RAAIMT	A predefined RFM threshold. RFM command is issued when the per-bank activation count reaches RAAIMT.
t_{RFM}	The duration of processing the RFM command.

In this dissertation, however, we focus solely on RowHammer-based DRAM read-disturbance errors.

Recently, a refresh management (RFM) interface was introduced in DDR5, especially for RowHammer-mitigation purposes [54,55]. Following this interface, a small per-bank ACT counter (called RAA count) exists at the memory controller (MC). When the ACT count reaches a predefined threshold (RAAIMT), the MC issues an RFM command to the corresponding DRAM bank, provisioning a predefined time margin (t_{RFM}). Table 2.1 summarizes the terminologies and abbreviations.

RowHammer Attacks: To exacerbate the RowHammer phenomenon, typical RowHammer (RH) attacks consist of three steps: 1) *memory templating*, *memory massaging*, and *bitflip triggering* steps. Figure 2.2 demonstrates the three steps of typical RowHammer attacks. First, in the memory tempating step,

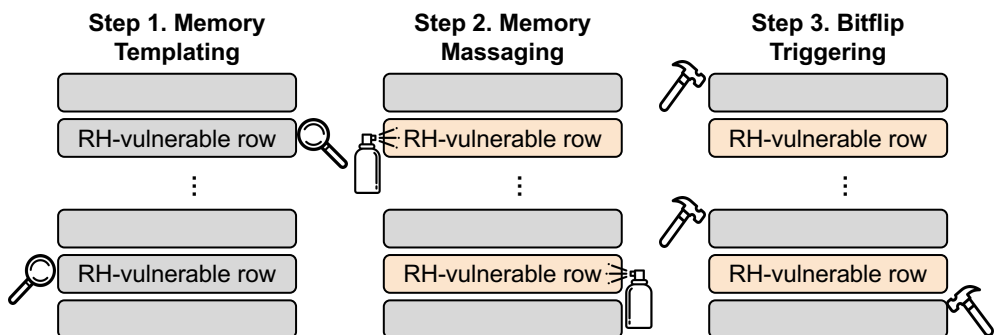


Figure 2.2: Three steps of a typical RowHammer attack. An attacker scans and collects the RowHammer-vulnerable DRAM rows (Step 1), sprays critical data objects to the corresponding rows (Step 2), and frequently accesses to trigger bitflips (Step 3). RH stands for RowHammer.

an attacker identifies RowHammer-vulnerable DRAM rows and their bitflip locations and gains control over the DRAM rows that are to be repeatedly accessed [23, 70, 104, 135, 137]. An attacker can infer the adjacency of DRAM rows by leveraging previously disclosed information [41] or by reverse-engineering static physical address to DRAM address (PA-to-DA) mappings [104], allowing the attacker to accelerate the memory templating process.

In the second (memory massaging) step, the attacker exploits the system’s memory allocator to force the victim process to allocate the objects in specific locations found in the memory templating step. Exploiting the information from the memory templating step, memory massaging techniques [23, 70, 74, 112, 135, 137] are often used to allocate the desired data on a vulnerable victim row.

Lastly, in the bitflip triggering step, the attacker generates memory access requests repetitively to trigger RowHammer bitflips. According to the DRAM

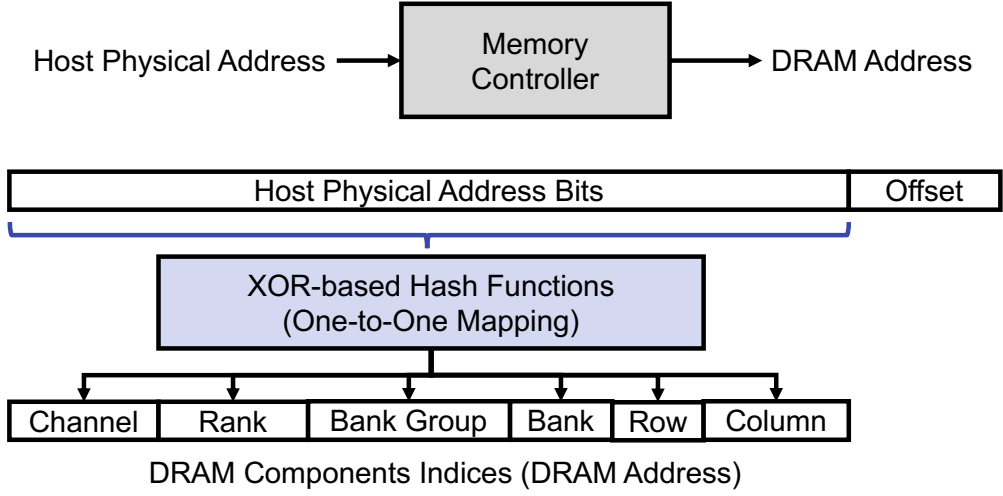


Figure 2.3: Overview of DRAM address mappings. The memory controller (MC) translates host physical addresses into corresponding DRAM addresses, represented as tuples of DRAM component indices. The mapping is one-to-one.

types and the system’s DRAM configurations, the attacker generates sophisticated memory access patterns [23, 29, 36, 49, 70] that cause hammering on the victim row (*e.g.*, double-sided attacks, many-sided attacks, and blast-attacks), sometimes even bypassing [23, 29] or abusing [70] any currently implemented RowHammer protection scheme [36].

2.4 DRAM Address Mappings

The MC translates the received physical address (PA) from a host processor to a specific DRAM device address (DA). Figure 2.3 demonstrates the overview of DRAM address mappings. A memory request is sent to specific DRAM cells based on the processor-specific way of dividing the PA into a tuple of channel,

rank, bank, row, and column indices, allowing the processor to exploit parallelism [2, 21, 25, 41–43]. When a PA arrives at a DRAM device, more complexity arises because DRAM manufacturers exploit additional spare rows and columns to remap faulty cells during the post-manufacturing process [51, 55] for better resilience [129]. Despite these complexities in physical address to DRAM address mapping, these issues are mostly *static* in nature.

XOR-Based Hash Functions. Modern MCs typically use *XOR-based hash functions* to translate physical addresses into DRAM component indices [40, 139, 144]. Each XOR-based hash function generates a single-bit hash value by XORing a selected subset of physical address bits. Thus, each function can be represented as a bitmask, where each set bit indicates that the corresponding address bit is involved in the hash function [139]. The complete mapping can be represented as a system of linear equations over $GF(2)$, often implemented as a binary matrix, designed to distribute memory accesses evenly [40, 139, 144].

Reverse-Engineering DRAM Address Mapping. Reverse-engineering these undocumented DRAM address mapping functions relies on observing system behavior. Prior methods employ brute-force [50, 88, 104] or educated-guessing [40, 144, 146] techniques, primarily analyzing row-buffer conflicts to deduce relationships between physical address bits and bank/row mapping. They generate random address pairs, observe conflicts (or lack thereof), and attempt to solve the underlying linear system with given input-output pairs.

While useful for identifying bank and some row/column bits, these conflict-based approaches generally cannot decompose the mapping into functions corresponding to memory systems’ architectural components (*e.g.*, channel, rank,

and bank group). Determining this full component-level decomposition is essential for precisely modeling memory behavior and enabling advanced security analyses, motivating the exploration of additional timing channels in this paper.

Chapter 3

Preventing RowHammer in DRAM with Intra-Subarray Row Shuffling

3.1 Overview

As RowHammer attacks have been a critical threat to computer systems, numerous hardware-based mitigation strategies for RowHammer have been proposed. However, the advent of non-adjacent RowHammer attacks and lower

This chapter is based on the following published and submitted work: [150].

- Minbok Wi, Jaehyun Park, Seoyoung Ko, Michael Jaemin Kim, Nam Sung Kim, Eojin Lee, and Jung Ho Ahn,
“*SHADOW: Preventing Row Hammer in DRAM with Intra-Subarray Row Shuffling,*”
In *Proceedings of the 29th IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.
© 2023 IEEE. Reprinted with permission.

RowHammer thresholds (Hammer Counts) significantly increases the area and performance overhead of these prior solutions due to their conservative design characteristics.

This chapter presents a new in-DRAM RowHammer protection solution named **Shuffling Aggressor DRAM Rows** (SHADOW). SHADOW dynamically randomizes DRAM row mapping information, preventing an attacker from targeting a specific victim row that may hold critical data. SHADOW is robust against non-adjacent RowHammer attacks because it utilizes the in-DRAM row-shuffle technique. To realize the in-DRAM row-shuffle operation with low performance and energy overhead, we use a novel DRAM microarchitecture optimization technique. We also utilize the recently introduced JEDEC RFM interface to enable in-DRAM RowHammer mitigation without requiring any changes in the DRAM interface. By exploiting an additional DRAM row per subarray, SHADOW does not require costly SRAM- or CAM-based tracking structures other than intrinsic counters for the RFM interface. We demonstrate the strong probabilistic protection of SHADOW against RowHammer attacks through adversarial pattern analysis and highlight the compelling performance, area, and energy overheads compared to those of state-of-the-art hardware-based RowHammer prevention solutions.

3.2 Threat Model

We assume the following threat model for designing SHADOW. First, we follow the existing RowHammer threat model that more than H_{cnt} activations on the

aggressor row within the refresh window (t_{REFW}) cause bitflips in the corresponding adjacent victim rows. Second, we assume more powerful RowHammer attacks that the aggressor row also affects non-adjacent rows within a short distance (blast-attack), where the effect is halved as the distance increases [64, 159]. Third, the specific subarray’s aggressor row does not affect the rows of other subarrays [19, 82, 83, 93, 94, 158]. Lastly, we assume the powerful RowHammer attacker who can control the physical address of the system and is aware of the initial static mapping of the DRAM rows (i.e., physical adjacency information between rows) [36, 49, 70, 102, 104, 146]. Therefore, the attacker can perform a sophisticated RowHammer attack by exploiting the effect of the blast radius.

3.3 Related Work

This section describes prior hardware-based RowHammer mitigation schemes and their mitigation against newly introduced RowHammer blast-attacks. We categorize them in terms of their *mitigating action* and *implementation locations* while focusing on their robustness against blast-attacks and their counter structures.

3.3.1 RowHammer Mitigating Actions and Blast-Attacks

Three major RowHammer mitigating actions have been introduced as a means of RowHammer protection for hardware-based RowHammer protection schemes: 1) an additional refresh of the victim row (Target Row Refresh), 2) throttling, and 3) row-shuffling.

TRR (Target Row Refresh) scheme refreshes the target/specific row to

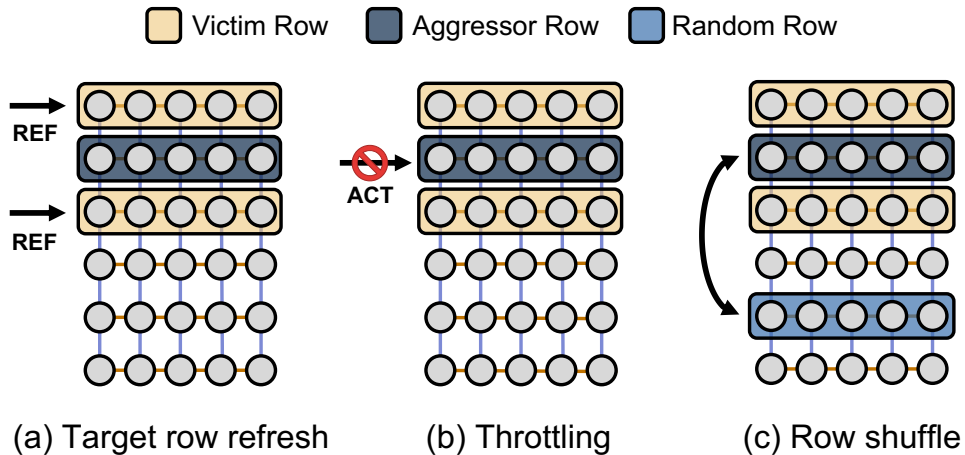


Figure 3.1: Three representative hardware-based RowHammer mitigating actions

recharge DRAM cells, which prevents the RowHammer bitflips if executed at the proper time based on tracking mechanisms [77, 101, 122] or at a high enough frequency based on random selection [68] (Figure 3.1 (a)). TRR can be implemented using the recently introduced RFM command [66, 86] or by modifying the conventional MC-to-DRAM interface [101]. However, recently introduced RowHammer fuzzers [29, 49, 50] automatically find the optimal RowHammer attack patterns that can bypass commercial DRAM’s TRR logic.

Throttling scheme modifies the MC scheduler to identify the thread showing a suspicious memory access pattern [12, 159]. When the hazard is recognized, this method limits the ACT frequency of the corresponding thread or DRAM row by adding delays (Figure 3.1 (b)).

While RowHammer protection schemes based on TRR or throttling possess unique strengths and weaknesses, they share a critical problem known as the

blast radius. As the physical address to DRAM address (PA-to-DA) mapping is static, sophisticated blast-attacks can intentionally degrade protection capabilities. Therefore, prior works require modification to their vanilla versions by decreasing the target H_{cnt} [116, 159] and sometimes even necessitating multiple mitigating actions for each event [66, 101], leading to exacerbated area, performance, and energy overhead.

Row-shuffle, a recently proposed RowHammer mitigating action, can be a solution for such a drawback [35, 115]. It identifies or samples an aggressor row and swaps its location in the DRAM address with another randomly chosen row in the DRAM address [115] or an additional copy row [35] (Figure 3.1 (c)). Multiple rounds of the row-shuffle operation can *dynamically* randomize PA-to-DA mapping and the row adjacency. This randomization prevents an attack from targeting a specific victim row, minimizing the possibility of double-sided attacks or blast attacks being successful and thus boosting the protection capability. Moreover, the memory templating step, as described in Section 2.3, cannot be undertaken successfully in such cases. Thus, well-known fatal attacks [29, 36, 49, 133] cannot be easily conducted as the attacker does not know which aggressor rows in the physical address to target to induce RowHammer bitflips on a specific target victim row.

However, existing row-shuffle-based solutions incur serious performance overhead originating from their inefficient row-shuffle operations. For example, the row-shuffle of RRS [115] blocks the corresponding memory channel from processing normal memory requests for 4,000 nanoseconds or more. This memory channel blocking incurs severe increases in the memory access latency and

bandwidth, which is unacceptable for latency-critical workloads [78]. Additionally, RRS does not fully utilize the PA-to-DA mapping randomization property of row-shuffle.

3.3.2 Implementation Locations and Counter Structures

Most state-of-the-art RowHammer protection schemes [66, 101, 109, 115, 159] require some form of an SRAM- or CAM-based counter structure to track the number of ACT commands, which grants them lower performance overhead at the cost of the area overhead. The MC-side implementation [101, 115, 159] allows one to take advantage of fast and dense transistors offered by a superior logic process and a large area budget. However, as one MC manages multiple DRAM devices, the total number of required counters must be set pessimistically. For example, RRS [115] requires a 43 KB SRAM per bank, resulting in more than 20 MB SRAM per processor, when considering 16 DDR5 ranks, which comes close to the last-level cache size. Unlike the MC-side implementation, DRAM-side implementation [66, 86] has optimization potential if considering chip-specific characteristics. However, DRAM-side implementation is limited by the stringent area budget and requires duplicate counter structures for each DRAM device. Most prior works encounter the scalability challenge [101, 109, 115, 159] of substantial chip area overhead of the counter structure under the recent trend of more DRAM channels, banks, lower H_{cnt} values, and a wider blast radius.

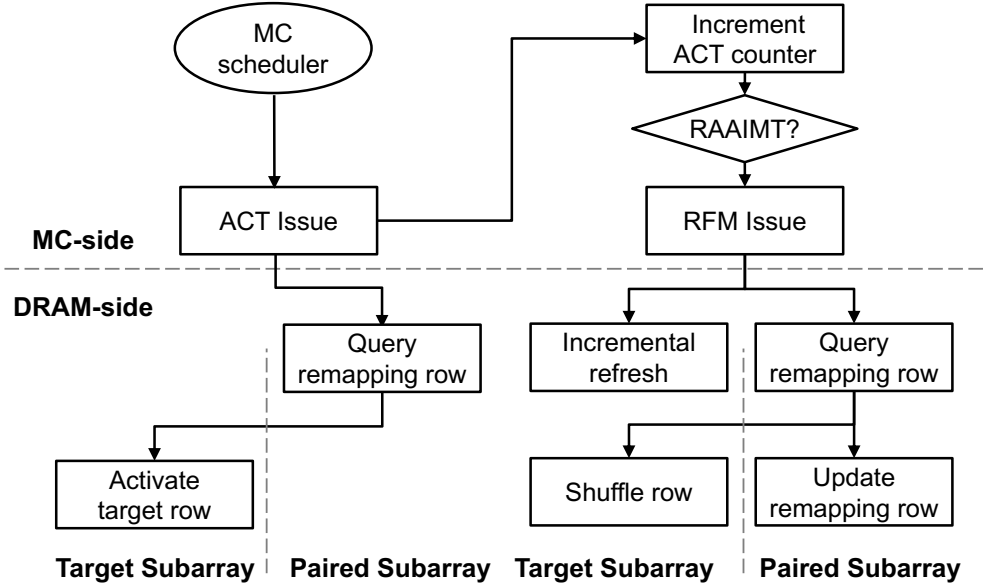


Figure 3.2: Operational flows of SHADOW.

3.4 SHADOW

3.4.1 Overview

SHADOW mitigates RowHammer by dynamically randomizing the spatial adjacency between aggressor and victim rows via an in-DRAM row-shuffle operation without needing a costly additional counter structure or long-latency memory channel blocking. First, to track the physical address to DRAM address mapping internally, SHADOW stores remapping information by populating an additional row, termed a remapping-row, inside each DRAM subarray. Also, SHADOW does not use a costly SRAM-based ACT tracking table to identify an aggressor row but simply selects random rows to shuffle. Second, the in-DRAM row-shuffle operation takes place upon every RFM command, leveraging the

intra-subarray row-copy operation proposed in prior studies [15, 121] to perform a high-throughput row-shuffle *inside a subarray* without a data transfer over the MC. To realize the secure and low-overhead RowHammer-protection scheme inside DRAM, we also leverage novel *incremental refresh* and *subarray-pairing strategies* as well as other previously suggested techniques. Their implementation details are discussed in Sections 3.5 and 3.6.

Figure 3.2 demonstrates the operational flow of SHADOW. At every ACT command, SHADOW refers to the remapping-row to translate the physical address to DRAM address (PA-to-DA). When the ACT count of a specific bank reaches RAAIMT, the MC issues an RFM command providing τ_{RFM} slack time for DRAM. Upon receiving the RFM command, SHADOW executes the in-DRAM row-shuffle operation targeting an aggressor and the incremental refresh strategy.

Table 3.1 summarizes the key terminologies and abbreviations used throughout this chapter.

3.4.2 High-Level Row-Shuffle Operations

Three different rows participate in a single row-shuffle operation: an aggressor row (Row_{aggr}), a randomly selected row (Row_{rand}), and an empty row (Row_{empt}). Row_{aggr} is the main target of the row-shuffle, which is randomly selected among recent RAAIMT numbers of activated rows. Row_{rand} is another randomly selected row in the same subarray of Row_{aggr} , which will be shuffled along with Row_{aggr} , providing randomness for SHADOW. Row_{empt} serves as a temporary buffer for an efficient row-shuffle with a dynamically changing DA

Table 3.1: SHADOW-specific terminologies and abbreviations

Symbol	Description
H_{cnt}	Hammer Count. The minimum activation count for incurring the bit-flip by hammering the row.
t_{RDRD}	An MC-defined timing parameter for consecutive read operations.
t_{REFI}	An average timing interval between two auto-refreshes.
$t_{\text{RFC}}, t_{\text{RFC}_{\text{sb}}}$	Refresh cycle for all bank refreshes and same bank refreshes, respectively.
RAAIMT	A predefined RFM threshold. RFM command is issued when the per-bank activation count reaches RAAIMT.
t_{RFM}	The duration of processing the RFM command.

location that is unreachable by the MC.

As depicted in Figure 3.3, two different row-copy operations are necessary for each row-shuffle operation at every RFM command. First, Row_{rand} is row-copied to Row_{empt} . Second, Row_{aggr} is copied to the previous DA location of Row_{rand} . Then, the previous DA location of Row_{aggr} now serves as Row_{empt} . After these movements, the remapping-row is updated accordingly to reflect the changed PA-to-DA location. This update preserves the functionality of SHADOW by redirecting the following memory accesses with the prior PAs to utilize the new DA locations. A series of row-shuffle operations leads to randomized PA-to-DA mapping, which changes dynamically at every RFM command.

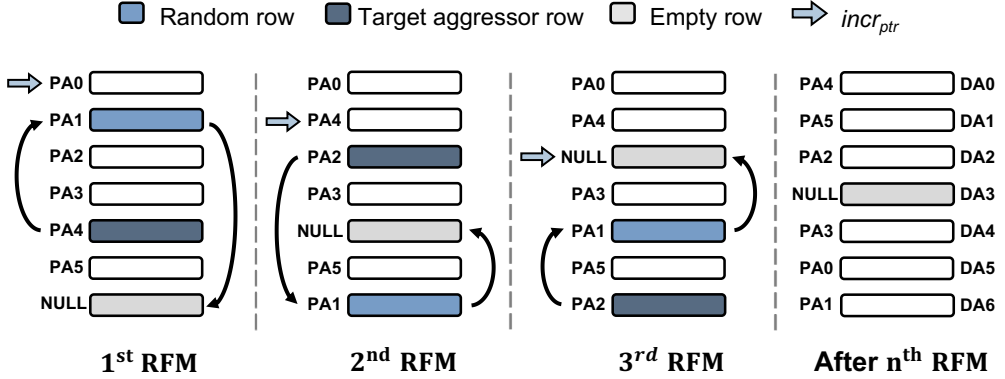


Figure 3.3: The operations of SHADOW on RFM commands, including row-shuffle and incremental refresh. After multiple RFMs, the PA (physical address) to DA (DRAM device address) mapping of each row is randomized.

3.4.3 Incremental Refresh

SHADOW also embodies a DA-based incremental refresh scheme to enhance protection capability by limiting the effective duration of an attack, especially on a frequently activated (possibly attacked) subarray. The incremental refresh executes an additional refresh on a row within the subarray, where the row-shuffle is executed, in a round-robin fashion upon the DA at every RFM command. This technique shortens the effective refresh window for each row from t_{REFW} (typically 64 ms in DDR4 or 32 ms in DDR5) to less than a millisecond, depending on the workload and SHADOW configuration. Therefore, this counterbalances the fact that the shuffling range of SHADOW is confined in a single subarray, which lowers the RowHammer protection capability compared to a larger shuffling space, such as a whole bank [115]. The detailed impact on security is elaborated in Section 3.7.1. The performance overhead of the incremental

refresh scheme is minimal because it only occurs once at every RFM command, and its latency is partially hidden via subarray-pairing (see Section 3.5.2).

3.4.4 Implications of Row-Shuffle in DRAM

When performing the row-shuffle operation, SHADOW relies on two distinct factors to achieve solid RowHammer-protection capabilities. First, SHADOW thwarts the concentration of ACTs in a single row by distributing ACTs to multiple rows in the subarray where ACTs frequently occur through row-shuffling, similar to the wear leveling operation of flash memory [16, 91, 145]. Also, in a subarray with a high ACT frequency, the effective number of ACTs in each row is reset to zero within a short time due to the incremental refresh operation. Although the shuffled rows can be accessed again by accident, it is difficult for the effective number of ACTs in the row to reach H_{cnt} within that short duration before being reset. We analyze the RowHammer-induced bitflip probability against several adversarial ACT patterns in Section 3.7.1.

3.5 SHADOW Hardware Architecture

SHADOW holds the physical address to DRAM address (PA-to-DA) remapping information of DRAM rows in an additional remapping-row, each specifically provisioned in every subarray. By confining the row-shuffling operation to a single subarray, SHADOW always tracks complete remapping information. We also propose subarray pairing, a microarchitectural optimization scheme that minimizes the timing overhead incurred when reading the remapping-row. Figure 3.4 demonstrates the high-level architecture of SHADOW.

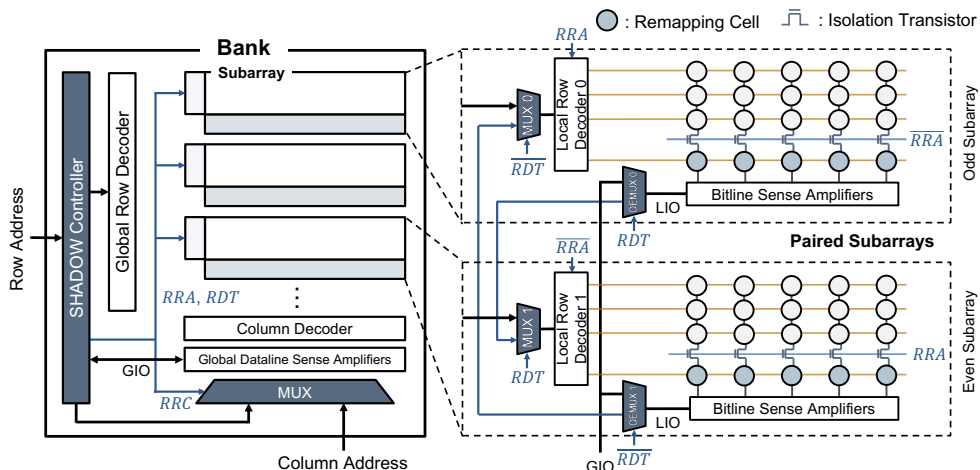


Figure 3.4: Hardware architecture of SHADOW

3.5.1 Remapping-Row

SHADOW stores the PA-to-DA mapping information in an additional row, called the remapping-row, within each DRAM subarray. The remapping-row stores the device addresses, and each is indexed by the physical address received from an MC. Based on the ordinary number of rows per subarray (512) and the size of a single row (1 KB), only $513 \times 9\text{-bit} = 577\text{ B}$ of storage is required per subarray as the row-shuffle of SHADOW is confined to a single subarray of 513 rows, including Row_{empt} . We also store the incremental pointer of each subarray in the remapping-row, occupying an additional 9 bits. Therefore, complete PA-to-DA mapping information can be comfortably held by a remapping-row 1 KB in size. We implement the remapping-row to be inaccessible by the MC, preventing any contamination of the PA-to-DA mapping.

To minimize the latency of accessing the remapping-row, we place it closest

to the row buffer and adopt isolation transistors, which decouple a remapping-row from the rest of the ordinary rows (right of Figure 3.4). The ACT latency is primarily affected by the difference in capacitance between a bitline and a cell [76,130], as a BLSA senses the change in voltage level caused by the sharing of cell charge with the bitline and amplifies it. An isolation transistor allows cells to share charges only with a short bitline whose capacitance is significantly lower than that of the original bitline when activating a remapping-row. Isolation transistors have already been used to share BLSAs across multiple bitlines in DRAM devices [30] and were exploited in previous studies as well [76,85,96,147], one of which reported an area overhead of 0.8%, including the supporting circuitry [96]. For the open-bitline structure [47] in which a single subarray accesses two different row buffers on the top and bottom, SHADOW can still preserve its access speed with two remapping-rows on both sides.

3.5.2 Subarray Pairing

To reduce the impact when accessing the remapping-row further, we propose a DRAM microarchitecture optimization technique, referred to here as *subarray pairing*. In SHADOW, every ACT accompanies an additional access to the remapping-row within the DRAM devices, increasing the ACT latency (t_{RCD}). If the target row of an ACT and the corresponding remapping-row are located in the same subarray, the ACT for the target row must wait until the remapping-row is precharged. SHADOW pairs every two subarrays (even and odd subarrays) and places the remapping data of each subarray into the remapping-row of its paired subarray (see Figure 3.4). With a minor modification, we can

concurrently activate a target row and a remapping-row that exist in different subarrays [69]. Then, a target-row ACT can be processed immediately after reading the remapping data without waiting for the PRE of the remapping-row in its pair. Therefore, the restore and PRE time of the remapping-row can be hidden by the ACT time of the target row.

Considering the open-bitline architecture in which two adjacent subarrays share a row buffer, SHADOW pairs two subarrays that sandwich another subarray between them (see Figure 3.4). Hereafter, the subarray containing the row to be activated by the MC or the row to be RowHammer-mitigated is defined as SA_{target} (*target subarray*), and the subarray having the remapping-row of the target subarray is defined as SA_{pair} (*paired subarray*).

SHADOW adds new signal paths to the subarrays to support subarray pairing. We add a remapping-row activation (RRA) signal to control the remapping-row ACT process that operates differently from other ACTs in the ordinary rows. The local row decoder receiving the RRA signal raises the wordline of the remapping-row, ignoring the address from the global row decoder. The inverse RRA signal turns off the isolation transistor to decouple the remapping-row from the long bitline. Moreover, to transfer the target row in the DA obtained from the remapping-row to SA_{pair} , we add new data paths between each subarray’s LIO and the local row decoder of SA_{pair} . We put a DEMUX and a remapping data transfer (RDT) signal to select whether the LIO is connected to the newly added path (for remapping-row access) or the GIO (for ordinary access). An added MUX determines whether the DA is accepted from the remapping-row or whether the address is accepted from the global row decoder.

The inverse RDT signal is used for SA_{pair} to avoid the simultaneous use of GIO.

3.5.3 SHADOW Controller

Each bank has a SHADOW controller that manages DRAM commands, addresses, and newly added signals according to the DRAM internal operation required for the target-row ACT and row-shuffling. First, this controller translates the row address in the PA of the ACT command transmitted from the MC to the corresponding column address in the remapping-row storing the target DRAM address. This column address transfers to the column decoder through a MUX controlled by the remapping-row-column (RRC) signal. The SHADOW controller also manages random numbers for selecting Row_{aggr} and Row_{rand} for row-shuffling and has latches for temporarily storing their addresses during the row-shuffling operation. The RNG (Random Number Generator) unit in each DRAM chip generates random numbers and securely buffers inside the SHADOW controller in advance to hide the random number generation latency. As a default, cryptographically secure PRNG (CSPRNG) based on the PRINCE block cipher [9, 114] is used. Further discussion, including the LFSR-based RNG option, is provided in Section 3.8.1.

3.6 SHADOW Detailed Operation and Timing

In this section, we analyze the operational flows of SHADOW for normal DRAM access and row-shuffle operation. We also calculate their latencies with the corresponding timing parameters.

3.6.1 Operation and Timing on ACT

SHADOW affects the normal DRAM access latency because fetching the remapping-row for the PA-to-DA mapping information must precede the processing of every ACT command from an MC (see Figure 3.5). It does not affect other DRAM commands, such as PRE, RD, or WR, because PRE does not require PA-to-DA mapping to precharge bitlines, and RD and WR target data in the row buffer.

The required time interval between ACT and the following RD or WR command is defined as t_{RCD} [51, 54, 55]. t_{RCD} can be broken down into multiple steps: i) command and address (C/A) traversal, ii) global row decoding, iii) local row decoding, and iv) activated row sensing. SHADOW requires three additional steps between ii) and iii): remapping-row decoding, remapping-row sensing, and remapping information (the target DA) traversal. We minimize the remapping-row decoding time using the RRA signal, as the location of the remapping-row is always fixed in a subarray. The remapping-row sensing time and the target DA traversal time are optimized using the isolation transistor and the newly added local data path between paired subarrays. We refer to this additional timing required to activate and read the remapping-row as t_{RD_RM} , and an ACT command in SHADOW requires t_{RCD}' (a modified form of t_{RCD}), which equals $t_{RCD} + t_{RD_RM}$.

3.6.2 Operation and Timing on RFM

We describe the detailed operation (Figure 3.6) and timing (Figure 3.7) of the row-shuffle and incremental refresh operations of SHADOW, which are executed

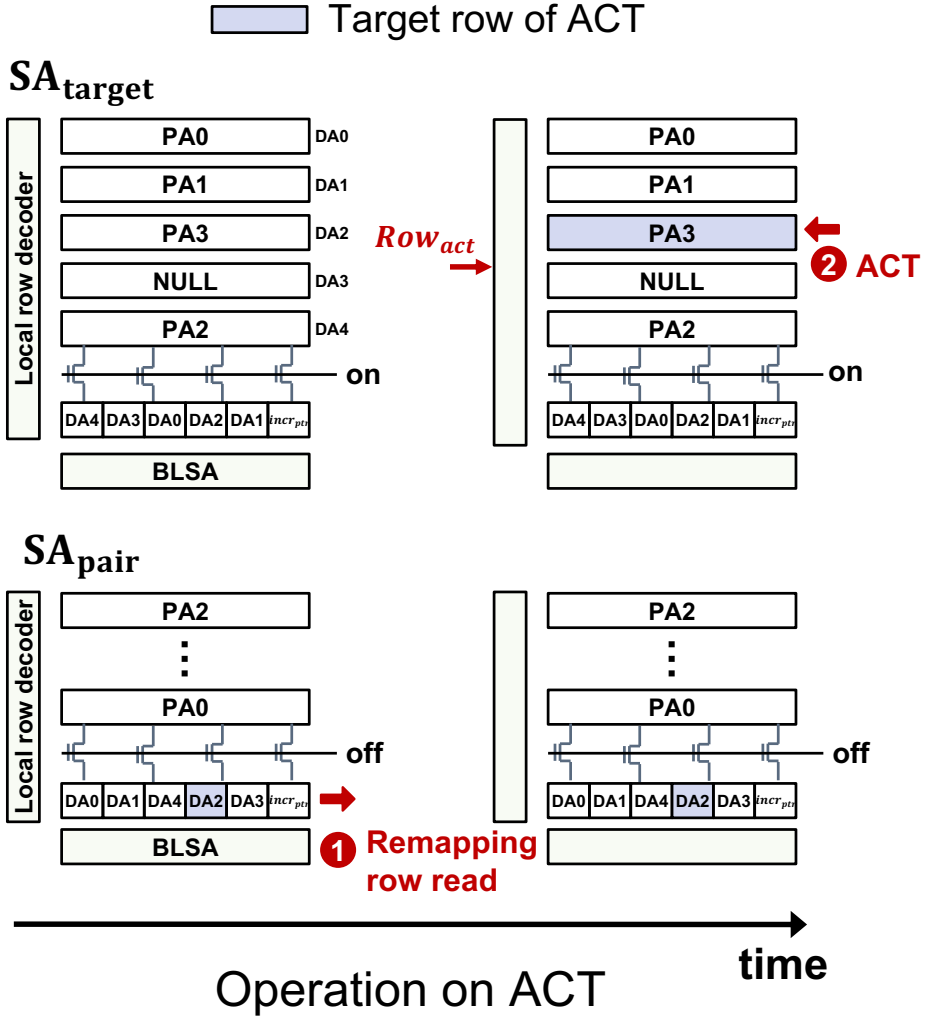


Figure 3.5: Detailed operational flows of SHADOW for the target subarray (SA_{target}) and its paired subarray (SA_{pair}) upon ACT commands.

upon every RFM command.

① **Remapping-row read:** Before initiating any operation at RFM, the remapping-row of SA_{pair} must be read to acquire four types of DA information: incr_{ptr} ,

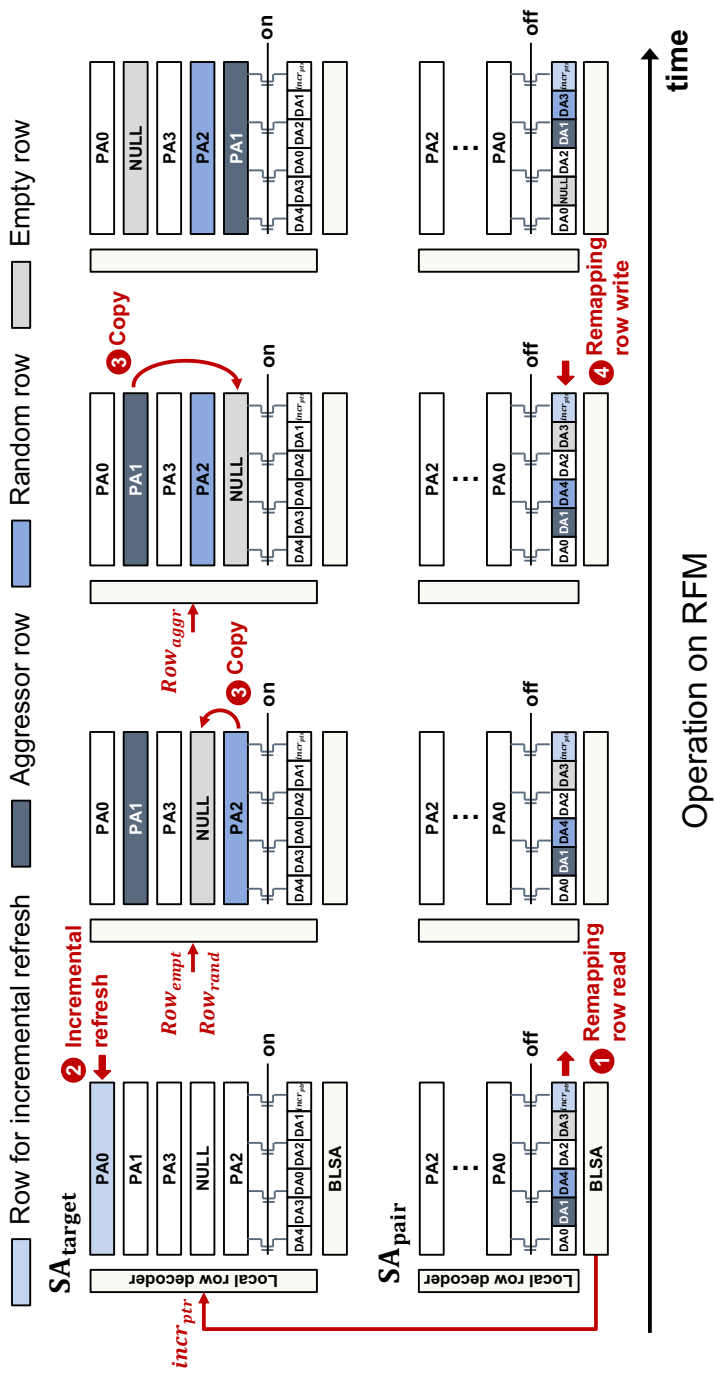


Figure 3.6: Detailed operational flows of SHADOW for the target subarray (SA_{target}) and its paired subarray (SA_{pair}) upon RFM commands.

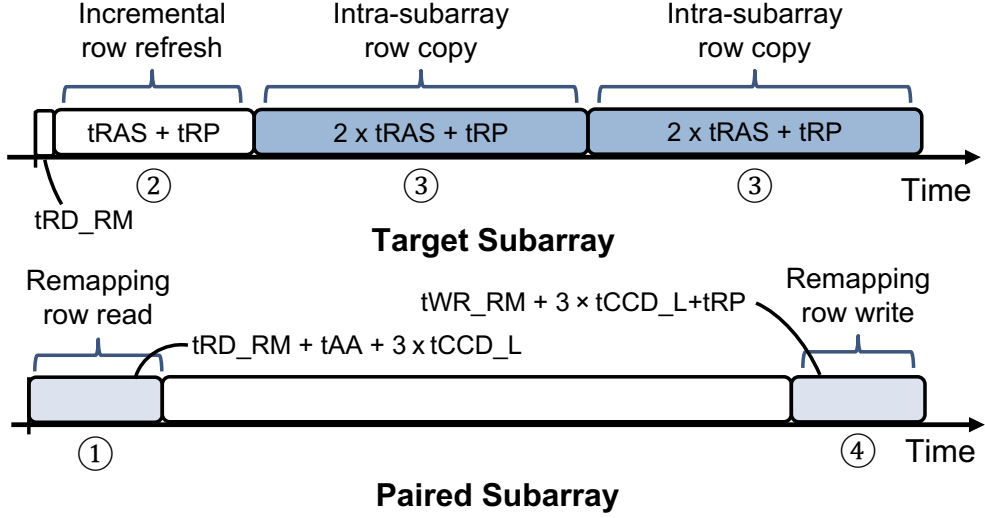


Figure 3.7: SHADOW operation timing diagram for the row-shuffle target subarray (SA_{target}) and the corresponding paired subarray (SA_{pair}) in a the RFM command.

Row_{aggr} , Row_{rand} , and Row_{empt} . The latency of activating the remapping-row and reading the first value ($incr_{ptr}$ in DRAM address) is t_{RD_RM} . After reading and transmitting the first value to SA_{target} , three consecutive reads are executed with proper PA column addresses, reading Row_{aggr} , Row_{rand} , and Row_{empt} into the latches of the SHADOW controller ($t_{AA} + 3 \times t_{CCD_L}$).

② **Incremental refresh:** After acquiring the $incr_{ptr}$ location, SA_{target} performs an incremental refresh by activating and precharging the pointed row. This process proceeds in parallel with the SA_{pair} , which is executing four additional reads. The incremental refresh ends with the additional ($t_{RAS} + t_{RP}$) time.

③ **Row-shuffle:** Following the incremental refresh, the row-shuffle operation in SA_{target} starts. It consists of two consecutive row copies. The DRAM addresses of Row_{aggr} , Row_{rand} , and Row_{empt} are already fetched by this time and stored in the SHADOW controller's latches during ①. The first row-copy step serves to sense the source row (Row_{rand}) to the row buffer, which uses t_{RAS} [121] to restore the row buffer fully. After the source row is latched, the wordline of the destination row (Row_{empt}) is driven to write back the data in the row buffer to the destination row, which again takes up to t_{RAS} . After a single row-copy, bitlines are precharged (t_{RP}), followed by the subsequent row-copy. Considering two row-copies, the overall execution time is $(4 \times t_{\text{RAS}} + 2 \times t_{\text{RP}})$.

④ **Remapping-row write:** In SA_{pair} , the remapping-row must be updated. The DRAM address values of $incr_{\text{ptr}}$, Row_{aggr} , Row_{rand} , and Row_{empt} are properly modified in the SHADOW controller and are written back to the remapping-row. Therefore, an additional internal activation for the remapping-row is needed ($t_{\text{RCD_RM}}$), followed by four consecutive writes ($t_{\text{WR_RM}} + 3 \times t_{\text{CCD_L}}$) and an internal precharge (t_{RP}) operation. $t_{\text{WR_RM}}$ refers to the write recovery time for the remapping-row. However, the latency of updating the remapping-row is *fully hidden* by the row-shuffle operation in SA_{target} . The calculated overall execution time is $(t_{\text{WR_RM}} + 3 \times t_{\text{CCD_L}})$.

In conclusion, the total execution time of the row-shuffle operation is as follows: $t_{\text{RD_RM}} + t_{\text{RAS}} + t_{\text{RP}} + 4 \times t_{\text{RAS}} + 2 \times t_{\text{RP}}$.

3.7 Evaluation

3.7.1 Security Analysis

We evaluate the protection capability of SHADOW by analyzing the Row-Hammer-induced bitflip probability, which reveals the SHADOW configurations for near-complete protection. We stipulate that a probabilistic scheme achieves near-complete protection when the bitflip probability of a DDR5-4800 rank with 32 banks is less than 1% per year, as in [66, 68, 101, 159]. The bitflip probability is analyzed with regard to the bitflip of *any victim row*, not a *specific victim row*. SHADOW prevents a bitflip of a specific victim row more strongly than prior RowHammer protection schemes that use static PA-to-DA mapping.

Defining adversarial patterns and attack scenarios: Upon our threat model, the attacker must rely on one of two different probabilities when forging an effective activation pattern against SHADOW. The former is the probability that the attacker guesses the new location of the shuffled row. The latter is the probability that there are multiple aggressors such that at least one of them does not become the target of a row-shuffle at every RFM until a bitflip occurs.

Attack scenario I exploits the former, where the attacker activates a single row within an RFM interval (i.e., the period between two consecutive RFM commands) and alters the PA of the row to be activated in the same subarray when a new RFM interval begins. As any row in the same array can be affected by this type of attack, we calculate the bitflip probability of this scenario while considering any bitflip in the subarray. This approach is similar to the birthday-paradox attack pattern in [115]. We exclude the effect where the rows

are refreshed due to ACT-PRE during a row-shuffle, thereby conservatively overestimating the possibility of a RowHammer bitflip.

Attack scenarios II and III take advantage of the latter probability, where the attacker activates multiple rows in the PA simultaneously. The attacker relies on the possibility that the activated aggressor will not become the row-shuffle target upon an RFM command. We again overestimated the RowHammer bitflip possibility by excluding cases in which the row-shuffle operation refreshes the victim row. The difference between scenarios II and III is that the former confines the aggressor rows in the PA within a single subarray, and the latter activates aggressor rows in the PA across multiple subarrays.

Calculating Bitflip Probability of Three Attack Scenarios: For analyzing the RowHammer-induced bitflip probability of SHADOW under three attack scenarios, we denote the number of rows in a single subarray as N_{row} (e.g., 512). We also define W_{sum} as the weighted sum of the effects that all aggressors within the blast range can have on a victim, setting it to 3.5 as the default value.

Attack scenario I: We represent scenario I by exploiting the buckets and balls model [115]; rows in the subarray and attack rounds are represented as buckets and balls, respectively. The number of buckets is N_{row} , as its definition suggests. Meanwhile, the number of balls is also N_{row} , due to the incremental refresh technique that limits the duration of attack scenario I under an incremental refresh window (i.e., N_{row} number of RFM commands). We define M_1 as the minimum number of RFM intervals required to activate the aggressor rows for an attacker to cause RowHammer bitflips on a victim row. At this

point, the bitflip probability of SHADOW for the attack scenario I, denoted as P_1 , can be described as the probability of one or more buckets containing M_1 balls when throwing N_{row} balls into N_{row} buckets. The success probability of one trial (p) is W_{sum}/N_{row} . We determine P_1 conservatively as follows using the Bernoulli trial method:

$$P_1 = N_{row} \times \binom{N_{row}}{M_1} \times p^{M_1} \times (1 - p)^{N_{row} - M_1} \quad (3.1)$$

For an *Attack scenario II*, we can create a recurrence formula for the bitflip probability of scenario II, denoted as $P_2[n]$, where n stands for the n -th RFM command. We denote the number of different aggressor rows that are activated as N_{Aggr} . We also define m as $RAAIMT/N_{Aggr}$, which means that m ACTs are executed for each aggressor during a single RFM interval. Similar to scenario I, M_2 denotes the necessary number of RFM commands by which an aggressor can evade a SHADOW row-shuffle to cause a RowHammer bitflip from the perspective of any single aggressor. The recurrence equation for $P_2[n]$ is as follows:

$$P_2[n] = P_2[n-1] + (1 - P_2[n-M_2-1]) \frac{1}{N_{Aggr}} \left(1 - \frac{1}{N_{Aggr}}\right)^{M_2} \quad (3.2)$$

We sweep all possible variations of N_{Aggr} , from 1 to $RAAIMT$. The incremental refresh of SHADOW sets an additional constraint for a successful attack; $m \times N_{row} < H_{cnt}$. This creates an upper bound of M_2 , decreasing the failure possibility. We calculate $P_2[N_{row}]$ and conservatively report $N_{Aggr} \times P_2[N_{row}]$, finding that there are N_{Aggr} different aggressor rows.

Table 3.2: RowHammer-induced bitflip probability of SHADOW for a DRAM rank within a year

RAAIMT	$H_{cnt} = 8K$	$H_{cnt} = 4K$	$H_{cnt} = 2K$
128	2E-15	4E-01	1
64	2E-43	1E-14	5E-01
32	0	1E-43	9E-15

*SHADOW's bitflip probability is calculated as the highest of the three attack scenarios.

Secure configurations for each H_{cnt} are marked in bold.

For an *Attack scenario III*, we again define M_3 with the same meaning, which is also expressed as H_{cnt}/m . The recurrence equation for the bitflip probability of scenario III, $P_3[n]$, is identical to Equation 3.2 only with M_2 replaced by M_3 . We conservatively ignore the possible benefit of an incremental refresh. With the last $P_3[last]$ bitflip probability in a **tREFW** window calculated, we again conservatively report $N_{Aggr} \times P_3[last]$.

SHADOW protection capability: Table 3.2 shows that SHADOW with an appropriate RAAIMT has sufficient protection capability in the form of a low bitflip probability even when H_{cnt} is as low as 2K. The detailed mathematical formula for calculating the bitflip probability considering the bank granularity and limited time interval (*e.g.*, **tREFW**) for each attack scenario is provided in Section 3.7.1; the values in Table 3.2 are the expanded probabilities of the rank granularity and the one-year time period.

Table 3.3: Timing values of SHADOW from the SPICE circuit simulation

Definition	Abbreviation	Timing	Baseline	Ratio
Row activation in SHADOW	tRCD'	17.7 ns	13.7 ns	+29%
Row copy w/ precharge	-	73.9 ns	-	-
Remapping-row sensing	tRCD_RM	2.3 ns	13.7 ns	-83%
Remapping-row write recovery	tWR_RM	9.0 ns	11.8 ns	-24%
Remapping-row read latency	tRD_RM	4.0 ns	13.7 ns	-71%

3.7.2 SPICE Circuit Simulation

We conducted a SPICE circuit simulation to measure the increased row activation time (t_{RCD}') and row-shuffle time in SHADOW.

Methodology: We referenced the methodologies in the prior works [35, 85] and the open-source SPICE DRAM circuit models [85] to derive the sensing time of the remapping-row. We built a DRAM subarray using 55 nm Rambus DRAM technology [111] with scaling to a 22 nm process based on the ITRS roadmap [45]. We modeled the sense amplifier and isolation transistor using the PTM high-performance model and modeled access transistors using the PTM low-power model [105]. We also modeled the local wordline driver by referring to prior DRAM modeling work [143]. To calculate the timing for DRAM address information reads from the remapping-row over to the paired subarray’s local row decoder, we modeled the new data path using the wire resistance and capacitance data in [111, 134].

Simulation results: Table 3.3 summarizes the SPICE circuit simulation re-

sults. It shows that $\mathbf{tRCD}'$, which contains the remapping-row ACT and RD time, is approximately $1.3\times$ of the original $\mathbf{tRCD}$. As mentioned in Section 3.6.1, $\mathbf{tRCD}'$ is the time obtained by adding $\mathbf{tRD_RM}$ to the default $\mathbf{tRCD}$. $\mathbf{tRD_RM}$ consists of the remapping-row decoding, remapping-row sensing, and remapping information traversal time. The decoding time in the local row decoder, which quickly turns on the wordline of the remapping-row with the RRA signal, is minimal at 0.33 ns. The timing for remapping-row sensing ($\mathbf{tRCD_RM}$) is less than 20% of the normal $\mathbf{tRCD}$ due to the isolation transistor. When the isolation transistor is turned off, the capacitance of the bitline for charge sharing with the remapping-row diminishes by more than $100\times$. Therefore, the sensing time of the remapping-row is 2.3 ns, resulting in a total time for the remapping-row ACT of 2.63 ns, whereas the baseline time is 13.7 ns. Also, the remapping information (DA) traversal time is less than 10% of the baseline $\mathbf{tAA}$. We conservatively set the DA traversal distance between two paired subarrays to half the bank size (*i.e.*, half-height and half-width) by referring to Samsung’s DDR4 [128]. In our SPICE simulation, the wire delay for DA traversal is less than 1 ns. This result is reasonable considering that the data traversal time of normal RD in previous works is less than 40% of $\mathbf{tAA}$ and that the DA traversal distance is a quarter of the original distance [125, 130].

We also analyzed the precise timing of the row-shuffle through a SPICE simulation. Inside the row-copy operation [121], we initially set the time required to copy the source row into the row buffer as $\mathbf{tRAS}$ to restore the row buffer fully, which is a conservative assumption. When SHADOW copies the value from the fully restored row buffer to a destination row, the necessary time is

Table 3.4: Actual machine system configurations for performance evaluation of SHADOW

Resource	Value
Processor	Intel Core i9-7940X @ 3.10 GHz
Org.	1 socket, 14 cores
L3 \$	19.25 MB, Shared
Main memory	4 channels, 1 DIMM per channel
DIMM type	DDR4-2666, 2 ranks, 16 GB
Timings (cycles)	19-19-19 (tCL-tRCD-tRP)
	467 (tRFC), 10400 (tREFI)

$0.55 \times t_{RAS}$, based on our SPICE simulation considering the small capacitance of the destination row compared to the bitline and row buffer. Therefore, the total operation time of the row-shuffle described in Section 3.6.2 is revised as $(t_{RD_RM} + t_{RAS} + t_{RP} + 3.1 \times t_{RAS} + 2 \times t_{RP})$, being 178 ns and 186 ns in the DDR4-2666 and DDR5-4800 configurations, respectively.

3.7.3 Performance Overhead Analysis

Our performance overhead analysis is two-fold: i) an actual system environment experiment that emulates an RFM interface and ii) an experiment based on an architectural simulator. The former allows us to understand the overhead of SHADOW and other RFM-based schemes (PARFM and Mithril [66]) on an actual system, which may be more capable of hiding the overhead of

RFM commands or the modified `tRCD` values. The latter enables comprehensive comparisons with additional state-of-the-art previous works (Mithril [66], BlockHammer [159], and RRS [115]).

Methodology: Table 3.4 summarizes the actual system configuration used for the experiments. Because a processor or a DRAM device supporting RFM commands is not yet publicly available, we emulated the RFM commands by increasing the frequency of conventional refreshes (by reducing `tREFI`). We modified the DRAM timing parameters (`tRCD` and `tREFI`) to emulate SHADOW using a modified Intel Performance Counter Monitor (PCM) tool. For the architectural simulations, we use McSimA+ [1] in a configuration matching an actual system environment, except for a change to DDR5-4800.

We used SPEC CPU2017 [11], GAPBS [6], and NAS Parallel Benchmark suites (NPB) [5] for performance evaluation of SHADOW. We categorized the applications in SPEC CPU2017 into three groups based on their memory access frequency: spec-high (bwaves, fotonik3d, lbm, mcf, wrf), spec-med (deepsjeng, gcc, xz), and spec-low (exchange2, imagick, leela). We constructed two multi-programmed workloads, mix-high consisting of 14 spec-high applications and mix-blend consisting of 14 applications uniformly chosen from spec-high, spec-med, and spec-low. Also, we used GAPBS traversing a Kronecker graph with 2^{26} vertices and NPB with class-C for multi-threaded workloads. For the simulation, we used mix-high and mix-blend workloads to populate the threads of 16 and 32 newly constructed mix-random workloads, which were created by randomly selecting 16 SPEC CPU2017 applications. We used the inverse of the execution time and the weighted speedup metric [28] to compare the performance of single-

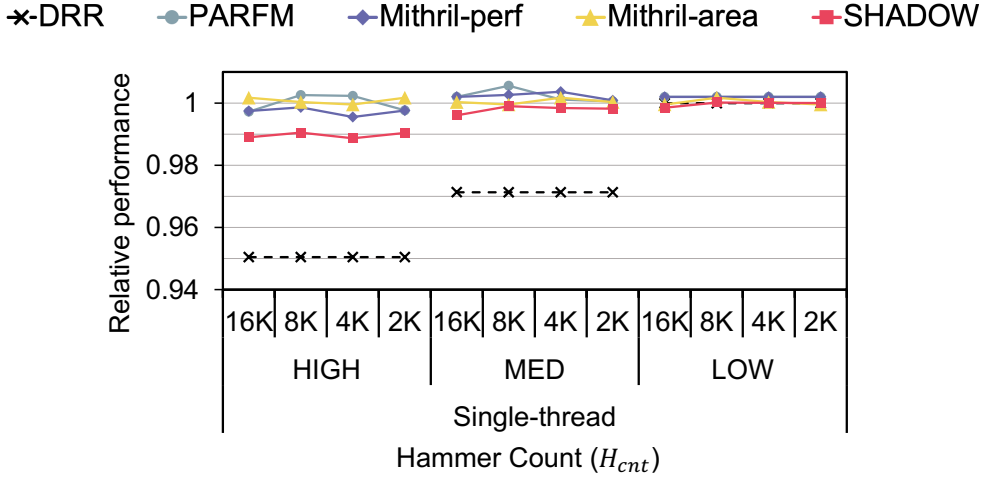


Figure 3.8: The relative performance of SHADOW, PARFM, Mithril-perf, Mithril-area, and the double refresh rate (DRR) compared to the baseline (the system without RowHammer prevention) on single-threaded SPEC CPU2017 on an actual system. HIGH, MED, and LOW indicate the average of the spec-high, spec-med, and spec-low applications, respectively. The performance represents the reciprocal of the execution time.

/multi-threaded and multi-programmed workloads, respectively.

We compared the SHADOW with PARFM, Mithril, RRS, and Block-Hammer. **PARFM** [66] is a probabilistic RowHammer-protection scheme (PARA [68]) with RFM. It performs TRR on a randomly sampled row whenever RFM commands are issued. We used the RAAIMT values of SHADOW and PARFM, resulting in a RowHammer-induced bitflip probability of 1% per year. **Mithril** [66] uses the CAM structure based on the Counter-based Summary (CbS) algorithm to track the ACT counts and the row addresses efficiently.

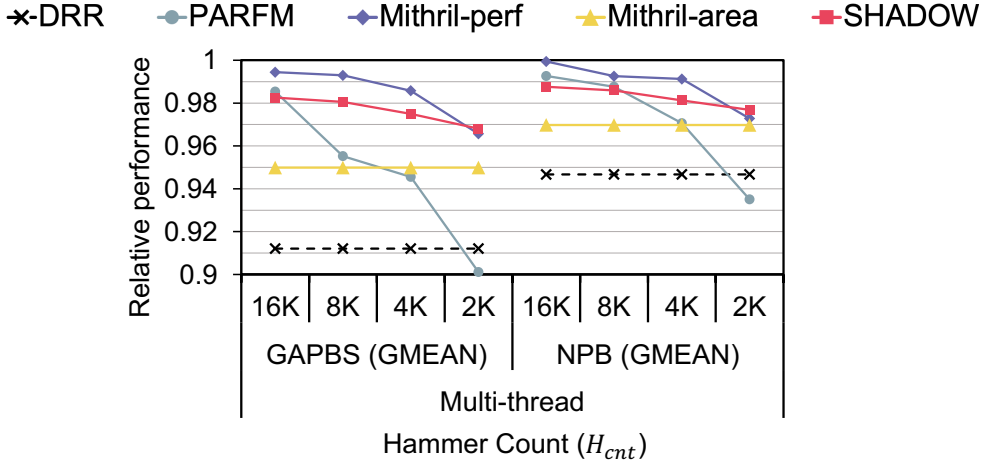


Figure 3.9: The relative performance of SHADOW, PARFM, Mithril-perf, Mithril-area, and the double refresh rate (DRR) compared to the baseline (the system without RowHammer prevention) on multi-threaded GAPBS, and NPB applications on an actual system. The performance represents the geomean of the reciprocal of the execution time for multi-threaded workloads. We used GAPBS using a Kronecker graph with 2^{26} vertices and NPB with class-C.

Mithril has the flexibility of setting different tracking table sizes and RAAIMT values for a given target H_{cnt} . We use two different classes of configurations for Mithril: performance-optimized Mithril (*Mithril-perf*) with 10KB per bank CAM table, which is expensive in terms of area and power for DRAM manufacturing technology [24, 34], and area-optimized Mithril (*Mithril-area*) with RAAIMT set to 32. *BlockHammer* [159], a representative throttle-based solution, is configured based on the memory access profiling results of the workloads and the target H_{cnt} . *RRS* [115], a state-of-the-art row-shuffle-based RowHammer mitigation scheme, is favorably configured with a row-swap threshold of 1/6

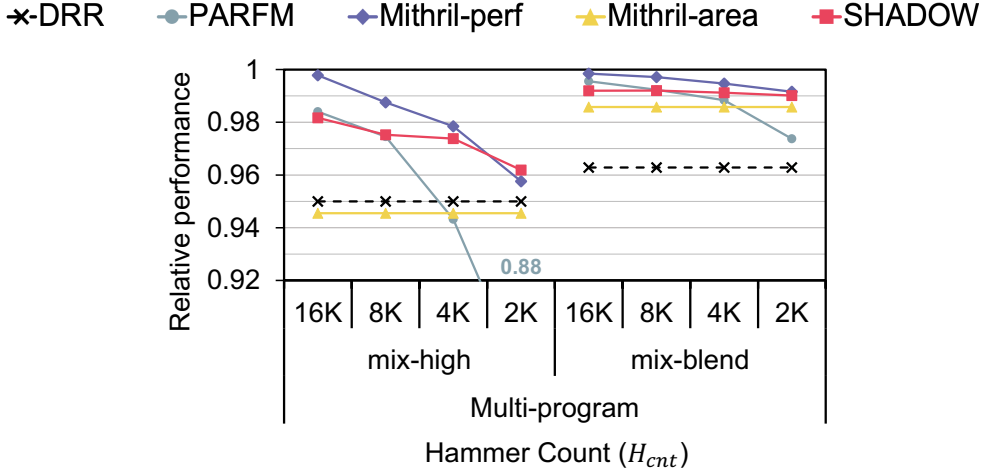


Figure 3.10: The relative performance of SHADOW, PARFM, Mithril-perf, Mithril-area, and the double refresh rate (DRR) compared to the baseline (the system without RowHammer prevention) on multi-programmed SPEC CPU2017 on an actual system. We constructed two multi-programmed workloads, mix-high consisting of 14 spec-high applications and mix-blend consisting of 14 applications uniformly chosen from spec-high, spec-med, and spec-low. The performance represents the weighted speedup for multi-programmed workloads.

of H_{cnt} . We consider the blast radius of 3 as the baseline; a recent study [64] reported that the blast radius could be as wide as 6.

RFM command emulation: We emulated the RFM interface by replacing RFM commands with additional auto-refresh commands. To calculate the desired number of auto-refreshes, we calculated the number of RFM operations during $\mathbf{tREFW}$ (N_{RFM}) by dividing the measured ACT counts by RAAIMT.

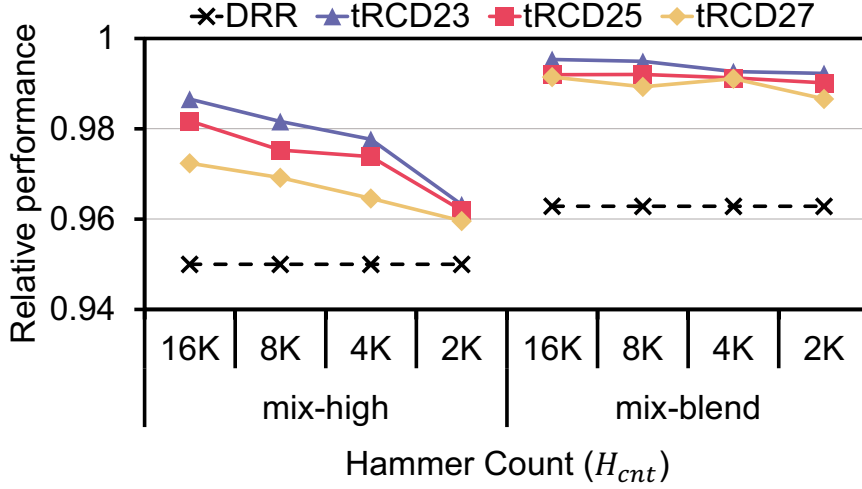


Figure 3.11: Relative performance (weighted speedup) of SHADOW with various tRCD parameters, tRCD23, tRCD25, and tRCD27, on mix-high and mix-blend multi-programmed workloads when varying H_{cnt} from 16K to 2K. Each weighted speedup is normalized to the baseline (tRCD19, the default tRCD parameter).

Then, we increased the number of additional refreshes during tREFW (N_{REF}) by reducing the tREFI parameter. We obtained the necessary value of tREFI' for each experiment through N_{RFM} and N_{REF} as the following equation:

$$tREFI' = tREFI \times \frac{tRFC}{tRFC + tRFM \times N_{RFM} / N_{REF}} \quad (3.3)$$

Actual system experimental results: Figure 3.8, 3.9 3.10 depict the relative performance of SHADOW on single-threaded, multi-programmed SPEC CPU2017 workloads and memory-intensive multi-threaded workloads (GAPBS and NPB) in comparison with the PARFM, Mithril, and double refresh rate

(DRR) approaches. First, when the system executes the single-threaded applications, SHADOW and the other schemes rarely increase the execution time. Despite the longer $\mathbf{tRCD}$ of SHADOW, the impact of SHADOW on the system performance is insignificant, being less than 2% even on spec-high. Even when H_{cnt} decreases, the change in such a trend is small because single applications incur neither a large number of ACTs nor RFM commands. With multi-threaded and multi-programmed workloads, SHADOW demonstrates comparable or superior performance compared to PARFM, especially when H_{cnt} is low. For Mithril-perf, the performance overhead is smaller than that of SHADOW, but it requires around 10KB CAM per DRAM bank. Lastly, the gap between Mithril-area and SHADOW shrinks at a lower H_{cnt} , but the required CAM structure of Mithril-area reaches 5 KB per bank when H_{cnt} is 2K. SHADOW generally shows less performance overhead due to its lower RAAIMT than PARFM and shows comparable performance compared to Mithril-area/perf.

Even under the worst-case synthetic, adversarial pattern of performance that is not observed in actual applications, SHADOW ensures robustness with less than 3% and 9% performance degradation compared to the baseline with and without the RFM command, respectively. The two main factors degrading SHADOW’s performance are the increase in $\mathbf{tRCD}$ and the periodic RFM command. Thus, we use a random stream microbenchmark [44] that issues frequent activations as the baseline. It is sensitive to $\mathbf{tRCD}$ changes and can frequently trigger RFM. Upon this, SHADOW’s $\mathbf{tRCD}$ increase results in a performance degradation of only 3%. Moreover, even when the effect of the theoretically most frequent RFM commands is imposed, calculated based on the maximum

possible number of activations under the JEDEC interface, less than 9% performance degradation is observed. Such an impact of the RFM is not only applied to SHADOW but also to all RFM-compatible schemes.

tRCD sensitivity in SHADOW: We conducted a sensitivity study of the tRCD timing parameters by changing the tRCD values to 23 tCK and 27 tCK from SHADOW’s default value of 25 tCK, where tCK is the DRAM cycle time (0.75 ns for DDR4-2666). Figure 3.11 illustrates the normalized weighted speedup in each case with the baseline parameter of 19 tCK tRCD without SHADOW (see Table 3.4). The performance effect of a higher tRCD value is noticeable when H_{cnt} is 16K, especially in the memory-intensive mix-high case. However, when H_{cnt} is as low as 2K, the effect of a different tRCD shrinks due to the more frequent RFM commands that are issued. In all cases, the performance overhead of SHADOW is less than 4%, which is much lower than doubling the refresh rate.

Blast radius sensitivity in SHADOW: SHADOW has less performance degradation than previous RFM-compatible schemes at a high blast radius. Figure 3.12 shows the performance overhead of SHADOW, PARFM, and Mithril according to the blast radius. When the blast radius is greater than two, SHADOW’s performance surpasses those of the other schemes.

Architectural simulation results: Figure 3.13 shows that SHADOW demonstrates robust performance in a wide range of workloads and H_{cnt} values, which is better than other schemes when H_{cnt} is less than 4K. In particular, the other row-shuffle-based RowHammer mitigation scheme (RRS) blocks the entire memory channel to perform row-shuffles, which incurs high performance

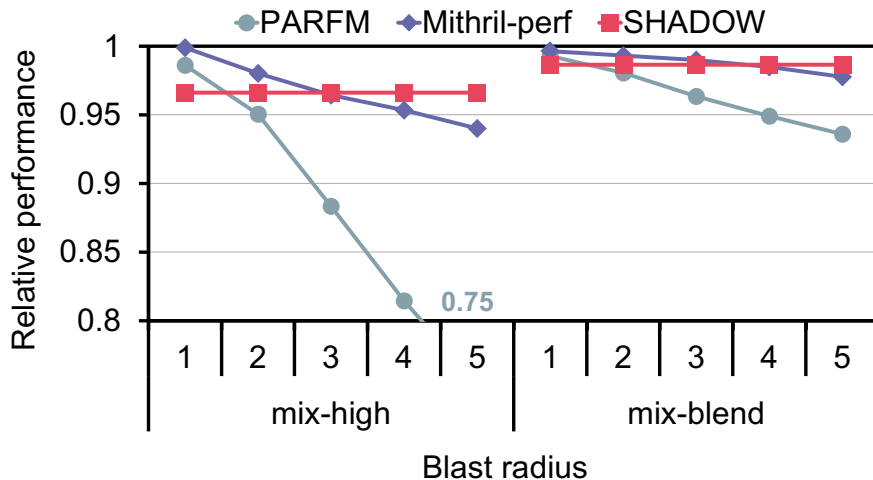


Figure 3.12: Relative performance (weighted speedup) of SHADOW when the blast radius ranges from 1 to 5 on mix-high and mix-blend multi-programmed workloads. RAAIMT is fixed at 2K. Each weighted speedup is normalized to the baseline (system without RowHammer mitigation).

overhead, as in [78, 120]. SHADOW’s row-shuffle operation utilizes the DRAM chip’s internal bandwidth, which is more efficient, especially when frequent row-shuffles occur at a low value of H_{cnt} . BlockHammer also incurs high overhead with a low H_{cnt} , as the necessary delay becomes too long [66, 109] and the possibility of misidentifying normal memory access also increases.

3.7.4 Area and Power Analysis

Methodology: To analyze the area and power overhead of SHADOW, we designed the SHADOW controller and additional logic in each subarray using Verilog. We synthesized them with CMOS 40 nm technology using the Synopsys

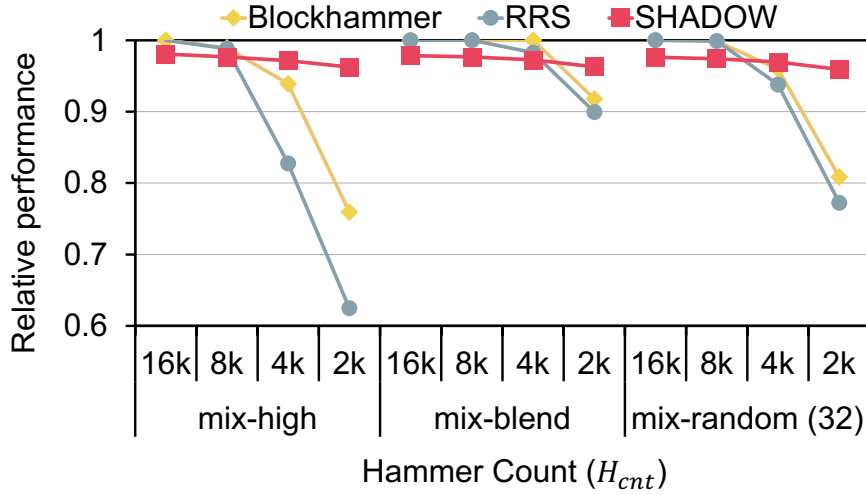


Figure 3.13: Relative performance (weighted speedup) of SHADOW with Block-Hammer [159] and RRS [115] in the mix-high, mix-blend, and 32 mix-random cases (which consists of randomly chosen SPEC CPU2017 applications) when sweeping H_{cnt} from 16K to 2K. Each weighted speedup is normalized to the baseline (no RowHammer mitigation).

Design Compiler [73]. We scaled the area overhead of the logic in SHADOW to a 22 nm DRAM process, considering that a DRAM process is $10\times$ less dense than an ASIC process due to the inferior driving current and fewer metal layers [24, 34]. We calculated the DRAM power using the Micron DDR4 calculator [89] and by referring to the Micron DDR5 datasheet [90]. In our experiments, we regarded the CPU’s TDP (Thermal Design Power) as the processor’s power.

Synthesis results: The total area overhead of the SHADOW logic in a DRAM chip is 0.35 mm^2 , which is equivalent to 0.47% of a DDR5 chip [63], and the DRAM capacity overhead for additional rows is 0.6% considering empty

rows and remapping-rows. These additional rows are feasible given the existing spare-row implementation scheme [61, 129]. The per-bank SHADOW controller includes an ACT counter, six 9-bit row address latches, a 7-bit subarray index latch, a MUX for a column decoder, and a control logic component. In each subarray, a MUX and a DEMUX are included. We assumed that the area overhead of three additional wire paths for the signals would be negligible compared to the number of wordlines and bitlines in a subarray. Also, these wire paths can be connected outside the DRAM cell array, resulting in negligible area overheads [164]. Lastly, SHADOW utilizes one PRINCE block cipher-based RNG unit per chip [9, 114, 115], which occupies less than 0.1% of the DRAM chip area, even when using the DRAM process.

The power consumption of SHADOW is negligible on the system-wide scale, being less than 0.63% even when H_{cnt} is 2K and with a memory-intensive workload. Figure 3.14 shows the relative system-level power consumption of SHADOW compared to that of the baseline system (system without RowHammer mitigation), with both calculated based on the number of memory commands (*e.g.*, ACT, PRE, RD, and WR) measured in the actual system environment using Intel PCM tools (Table 3.4). Based on the number of memory commands (ACT and PRE), we calculate the total number of RFM commands to issue during program execution. The number of issued RFM commands increases as H_{cnt} decreases. However, the fact that the overall power consumption of SHADOW changes minimally suggests that it is dominated by the additional remapping-row accesses rather than the row-shuffle operations.

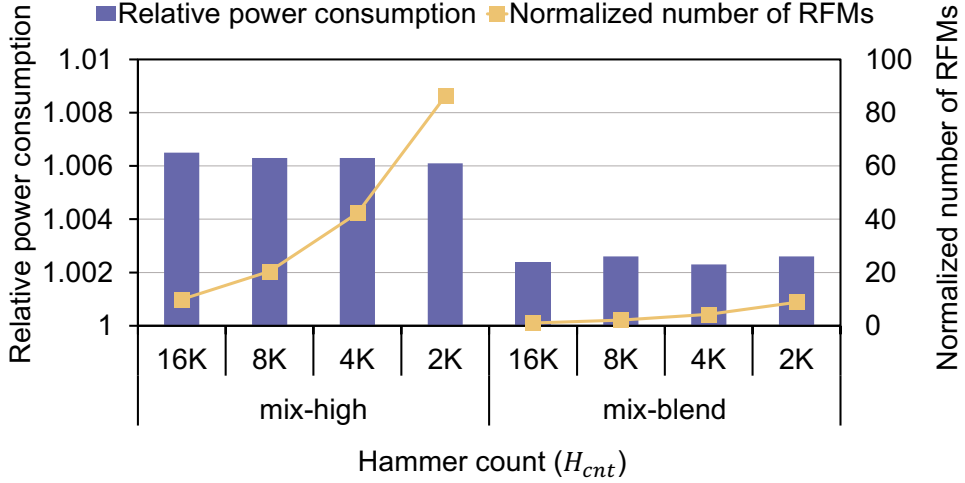


Figure 3.14: Relative system-level power consumption of SHADOW compared to the baseline (system without RowHammer mitigation) and the number of RFM commands normalized to the number of refreshes of the baseline on mix-high and mix-blend multi-programmed workloads across varying H_{cnt} values ranging from 16K to 2K.

3.8 Discussion

3.8.1 Hardware RNG Unit in DRAM

We can implement the random number generator (RNG) unit in DRAM with negligible overhead based on block ciphers, as in [9, 56, 114, 115]. The required per-chip random number throughput of SHADOW is 126 Mbits/s, with 4K H_{cnt} and a conservative 32-bit random number for Row_{aggr} and Row_{rand} in each case. The PRINCE block cipher [9] can generate more than 1 Gbit/s of random numbers, even under the slow DRAM core frequency. It can also be trans-

ferred via a 1-bit signal path in advance and buffered at each bank’s SHADOW controller. Additionally, a linear-feedback shift register-based (LFSR) RNG with its seed being periodically randomized can also be used to ensure much lower area/power overhead [103, 162]. Recent DDR5 memory chips are already equipped with LFSR to generate read training patterns [55].

The security of the RNG unit also affects the security of SHADOW. Therefore, the use of a cryptographically weak PRNG unit can expose the row-shuffle’s Row_{rand} , allowing the attacker to infer the physical address to DRAM address mappings. We can enhance the security of RNG units by utilizing boot-time or periodic key and counter initialization strategies that leverage CPU-side true RNG units.

3.8.2 Soft Post Package Repair (sPPR)

Since DDR4, the JEDEC interface defines an sPPR mechanism [51, 55] that allows an OS or a user to replace a faulty DRAM row address during runtime per each bank group. The fact that the tRCD value is unmodified even after sPPR implies that there is a low-overhead address relocation path that is reusable by SHADOW. Also, the number of possible sPPR replacements per bank group has continually increased over recent generations [113], suggesting that SHADOW can exploit such resources in the future or even provide the required mechanism for such an enhanced sPPR.

3.8.3 Optimizing RFM Interface

Earlier suggested efficient memory-access filtering structures can be orthogonally adopted to SHADOW by optimizing the RFM interface. Prior works, such as BlockHammer [159] and Hydra [109], utilize efficient random projection-based counter structures [92], including a dual-counting Bloom filter (D-CBF) or a group-count table (GCT), which enable them to filter out the majority of memory accesses from normal workloads. When such a filtering process exists before issuing the RFM command, it has the potential to significantly reduce the number of unnecessary RFM issues (false positives) and thus mitigate the ensuing performance degradation.

3.8.4 Per-Row Activation Counting (PRAC)

To address the growing threat of RowHammer attacks, JEDEC introduced Per-Row Activation Counting (PRAC), a mechanism that tracks the number of activations for each DRAM row [55]. PRAC seems an ideal tracking-based RowHammer mitigating solution as it maintains fine-grained activation counts on a per-row basis. When a row’s activation count exceeds a predefined threshold, the DRAM issues an Alert Back-Off (ABO) signal, triggering the memory controller to send targeted refresh or RFM commands. Despite its potential, recent studies have shown that RowHammer attacks remain feasible under PRAC implementations [13, 14]. For example, [107] exploits the limited capacity of the row-tracking queue to overflow the flip threshold (H_{cnt}), while [14] demonstrates how PRAC-triggered mitigations can be abused to launch denial-of-service (DoS) attacks. These findings highlight that, even with PRAC, RowHammer

remains an open challenge, necessitating continued research into robust and secure mitigation strategies [13, 65, 106, 107, 142, 151–153].

Chapter 4

Decomposing DRAM Address Mapping into Component Functions

4.1 Overview

Understanding how physical addresses are translated to DRAM components—such as channels, ranks, bank groups, and banks—is crucial for analyzing memory behaviors and conducting precise security analyses, including DRAM read-disturbance errors. However, previous reverse-engineering methods have lacked

This chapter is based on the following published and submitted works: [149].

- Minbok Wi, Seungmin Baek, Seonyong Park, Mattan Erez, and Jung Ho Ahn, “*Sudoku: Decomposing DRAM Address Mapping into Component Functions*,” arXiv preprint arXiv:2506.15918, 2025. Presented at DRAMSec 2025 co-located with ISCA 2025.

the capability for fine-grained decomposition of DRAM address mappings and the generality required for modern processors and their memory systems.

To address these limitations, we introduce novel timing-based techniques leveraging DRAM refresh intervals and consecutive access latencies to infer component-specific functions. Based on this, we present Sudoku, the first software-based tool to automatically decompose full DRAM address mappings into channel, rank, bank group, and bank functions while identifying row and column bits. We validate Sudoku’s effectiveness, successfully decomposing mappings on recent Intel and AMD processors. Lastly, we open-source Sudoku to facilitate future research on reverse-engineering DRAM address mappings and system-level memory analysis.¹

4.2 Related Work

This section describes prior studies on reverse-engineering DRAM address mappings and highlights the limitations of existing approaches. We focus on XOR-based hash functions, which are widely adopted in modern processors [31, 40, 104, 141, 144]. Their mathematical properties have been comprehensively explored in [139].

4.2.1 Reverse-Engineering DRAM Address Mappings

Modern memory controllers commonly use XOR-based hash functions for DRAM address mappings [19, 31, 40, 48, 139, 144]. Reverse-engineering XOR-based hash functions is similar to solving a system of hash functions, and a black-box approach has been widely used in prior studies [50, 88, 104]. This ap-

¹<https://github.com/scale-snu/sudoku.git>

proach typically involves two phases: 1) collecting and validating input-output pairs and 2) deriving the hash functions based on the observable behavior.

Methods for Collecting and Validating Data: Accurately generating and validating input-output pairs is essential for recovering the underlying hash functions. Prior studies rely on two types of measurement techniques: *indirect* and *direct* methods.

Indirect methods infer DRAM address mappings by observing latency variations in memory access. A widely used technique is exploiting DRAM row buffer conflicts [50, 87, 104, 146, 157]. When two addresses map to the same bank but different rows, a row buffer conflict occurs, leading to increased memory access latency. These latency differences provide a single bit of information on whether two addresses are mapped to the same bank.

Direct methods measure component-level DRAM activations directly, often requiring specialized hardware features. For example, reMap [38] uses hardware performance counters (HPCs) to detect component-level DRAM activation, while other studies use custom hardware, such as silicon interposers or oscilloscopes, to monitor DRAM signals [19, 48, 75, 104]. However, such methods are infeasible on modern systems with on-board or integrated DRAM, where physical access to the memory modules is restricted [21, 126].

Methods for Deriving Hash Functions: After collecting a sufficient number of validated input-output pairs, the next step is to infer the hash functions from the datasets. Exhaustively searching all possible functions is computationally infeasible because of the exponential complexity of the search space [31, 139]. Thus, prior works generally assume the use of XOR-based hash functions, as

these are linear and commonly used in microarchitectural components such as cache indexing and DRAM address mappings [40, 50, 88, 104, 144, 146].

Given that XOR functions are linear over $GF(2)$, reverse-engineering them equals solving a system of linear equations [139]. One common approach is brute-forcing (or exhaustively searching) [50, 104], which involves exhaustively generating and testing all possible candidate bitmasks to identify the correct XOR hash functions. Alternatively, a constraint-based (or educated guessing) approach [40, 146] reduces the search space by leveraging known DRAM addressing functions. This approach enables a more efficient identification of hash functions.

4.2.2 Limitations of Prior Reverse-Engineering Methods

While existing reverse-engineering tools for DRAM address mappings can successfully identify addressing functions, they have several limitations: (1) they do not precisely identify all row and column bits, (2) they cannot decompose DRAM addressing functions into component-level functions, and (3) they require physical access to the system or specific hardware support. Table 4.1 summarizes the capabilities and requirements of existing tools for reverse-engineering DRAM address mappings.

Need for Full DRAM Addressing Functions: Recent RowHammer fuzzers require full DRAM addressing functions—including row and column bits—to enable accurate system-level analysis of RowHammer attacks [29, 49, 50]. Also, identifying component-level functions (*e.g.*, channel, sub-channel, bank group, and bank address) is crucial for understanding performance characteristics and

Table 4.1: Comparison of capabilities of prior tools for reverse-engineering DRAM address mappings

Tool	Finding Functions	Identifying Bits	Decomposing Functions	Type	Hardware Dependency
DRAMA [104], DARE [50], RISC-H [87]	✓	△	△	SW/HW	Optional
Graph [157], DRAMDig [146]	✓	△	✗	SW-only	No
reMap [38]	✓	✗	△	SW/HW	Yes
MEMBUSTER [75]	✓	✓	✓	HW	Yes
mFIT [19]	✓	✓	✓	HW	Yes
Sudoku	✓	✓	✓	SW-only	No

security implications against recent RowHammer attacks [60]. However, prior work [38,50,88,104,146] has not effectively reverse-engineered these fine-grained mappings, limiting their applicability in detailed DRAM system studies.

Need for physical access or hardware support: While hardware-based approaches can reverse-engineer DRAM address mappings precisely, they require specialized equipment (*e.g.*, memory interposers or signal analyzers) or rely on processor-specific features. For example, mFIT [19] directly captures DRAM commands and data transfers using a custom memory interposer. Other works [48,50,75,104] employ high-frequency oscilloscopes to probe electrical signals. However, all of these methods require physical access to the system, which becomes infeasible when DRAM is soldered or integrated onto the motherboard [21,126]. reMap [38] leverages hardware performance counters to monitor fine-grained memory-related events; however, it still relies on processor-specific support for these counters and events.

4.3 Understanding How Systems Configure Memory

Understanding how the system configures memory and timings is crucial for accurately analyzing timing channels in DRAM-based memory systems. We

identify these by examining BIOS settings and reading processor-specific MC-related registers [2, 25, 42, 43]. For example, we obtain precise values for key timing parameters such as t_{RFC} , t_{REFI} , and various t_{RDRD} timings, which are essential for our subsequent analyses. We also examine the MC-related registers that are no longer documented in recent processors but were disclosed in the previous datasheets, as in [71], to obtain several configurations related to DRAM address mapping [42, 43].

Knowing the system-configured refresh interval (t_{REFI}) is particularly important, as it depends on the type of refresh operations and DRAM chip density [51, 55]. We also note that on the tested Intel processors with DDR5 (Table 4.4), the MC treats DDR5 sub-channels as two independent memory channels; a single-rank DDR5 module is perceived as two channels, and populating two physical channels results in the MC managing four logical channels. This understanding informs our timing analyses on systems with DDR5.

Modern processors also provide limited information about DRAM address mapping through MC-related registers [42, 43], enabling verification of specific DRAM component mapping functions. For example, Intel Core processors reveal a channel hash mask and the bit used for bank group selection through their MC-related registers [42, 43].

Figure 4.1 illustrates the process of reading MCHBAR registers in Intel processors. MCHBAR registers are accessible via memory-mapped input/output (MMIO), requiring each register’s physical address. To access MC-related registers, we first read the base address of MC-related registers from the PCI configuration region. We can calculate the actual MMIO address from the base

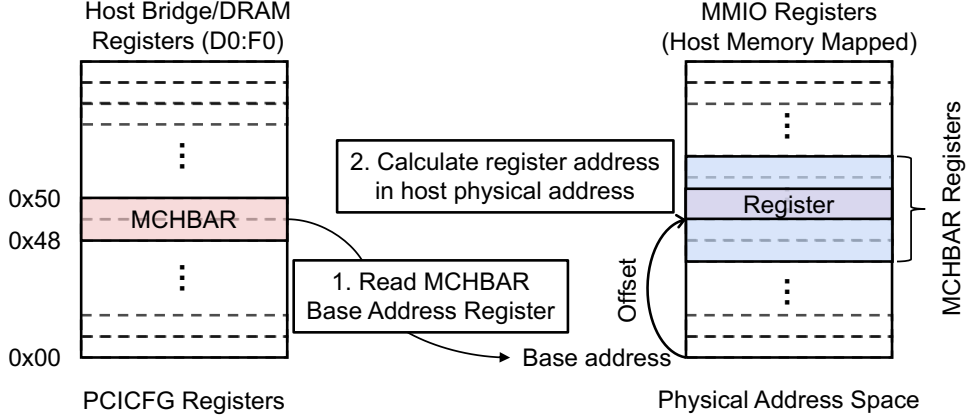


Figure 4.1: The process of accessing MCHBAR registers involves: 1) reading the base address from the PCI configuration space, and 2) calculating the MMIO-mapped physical address of the desired register.

Table 4.2: MCHBAR register information and MC timing parameters used in Sudoku

Register Name	Offsets	Related Fields
tRDRD	0xE00C	Diff DIMM, Diff Rank, Diff Group, Same Group
DIMM Characteristics	0xD80C, 0xD810	BG0 bit options
Channel Hash	0xD824	Hash mode, Hash LSB mask bit

address and the corresponding offsets of MMIO-mapped registers. From those calculated addresses, we can extract DRAM timing values, such as t_{RFC} , t_{REFI} , and t_{RDRD} , and identify DRAM address mapping-related fields. For example, these fields include BG0 bit options and hash masks, which are critical for validating and refining our inferred address mapping functions. Table 4.2 summarizes the name of registers, and their fields.

Also, in Linux, the EDAC (Error Detection and Correction) subsystem uti-

Table 4.3: Results of DRAM component-level timing channel analyses

Processor	DDR	Reduced Refresh Interval	Latency Comparison (Consecutive Accesses)
Intel Core 12th	DDR4	Ch, DIMM/RK	Ch < BG < DIMM/RK, BA
Intel Core 12th/14th	DDR5	Ch, SC, BA	Ch/SC < BG < BA < DIMM/RK
AMD Zen 4	DDR5	SC, DIMM/RK	Ch < BG < SC < BA < DIMM/RK

Abbreviations: Ch = Channel, SC = Sub-channel, BG = Bank Group, BA = Bank Address, DIMM/RK = DIMM or Rank.

lizes these MC-related registers to accurately diagnose the DRAM error locations, thereby revealing portions of DRAM address mappings in the server processors [80,82]. We discuss these mechanisms in more detail in Section 4.7.1. We leverage this DRAM address mapping information to validate Sudoku’s results (Section 4.6).

Finally, while accessing system registers aids our timing channel analysis, our Sudoku tool does not require it.

4.4 Timing Channels for Component Function Identification

In this section, we analyze DRAM timing channels beyond simple row conflicts, focusing on refresh operations and consecutive accesses to infer component-level mapping details. Table 4.3 summarizes the results of timing channel analyses for component function identification.

4.4.1 Refresh Intervals

Beyond simple detection, we exploit refresh-induced latency spikes to infer whether two addresses share the same refresh resources, thereby forming a refresh group. By alternately accessing a pair of randomly generated memory

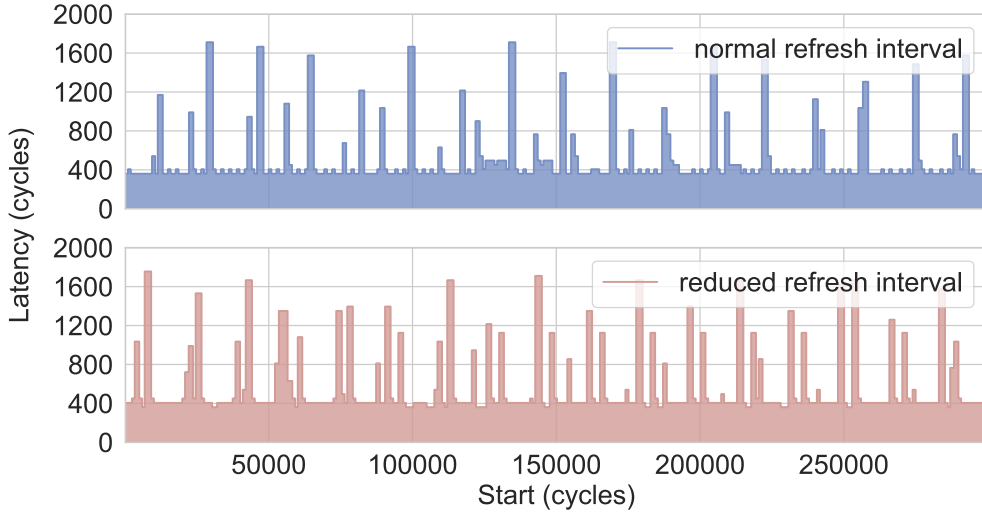


Figure 4.2: Measured refresh-induced latency spikes in the AMD Ryzen 9 7950X processor with DDR5 operating at 4.5 GHz. We iteratively access two memory addresses, measure their latencies, and compute the average spike interval. Two types of refresh intervals are shown: a normal refresh interval (top) and a reduced refresh interval (bottom).

addresses and monitoring the interval between periodic latency spikes [29, 50], we observe distinct patterns. Addresses mapped to the same refresh group yield spikes at the standard t_{REFI} interval. By contrast, addresses mapped to different refresh groups produce two interleaved spike trains, resulting in a reduced effective interval between observed latency spikes. Figure 4.2 shows measured memory access latencies over time in the AMD Ryzen 9 7950X processor with DDR5. For the normal refresh interval, we observe monolithic and periodic latency spikes. In contrast, for the reduced refresh interval, there are two distinct periodic spikes with a certain offset. This directly reveals whether the address pair differs in the address bits determining the refresh group.

Applying this method, we observed all-bank refresh on an Intel 12th Gen

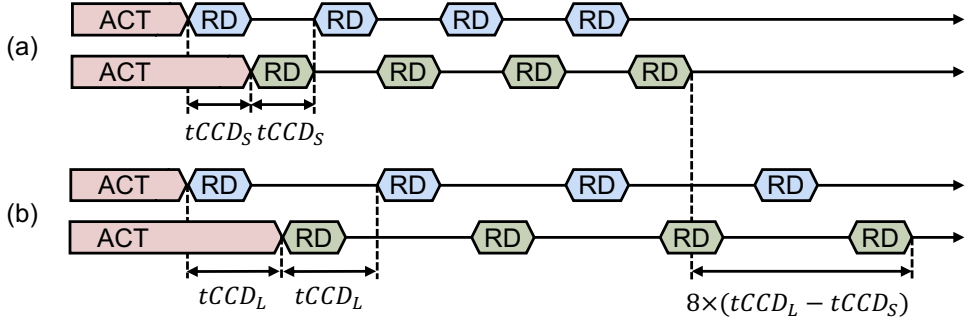


Figure 4.3: Theoretical timing differences between two consecutive memory reads. Consecutive reads exhibit different timing characteristics depending on their mappings. (a) Reads to different bank groups ($tCCDS$) incur a shorter interval than (b) reads to the same bank group ($tCCDL$).

processor with DDR4, fine-grained refresh on Intel 12th (Alder Lake) and 14th (Raptor Lake Refresh) Gen processors with DDR5, and all-bank refresh operations on an AMD Ryzen Zen 4 processor with DDR5.² Furthermore, since XOR-based hash functions tend to partition the address space evenly, the proportion of random address pairs exhibiting reduced intervals provides an estimate of the number of address bits (and thus, XOR functions) involved in determining the refresh group.

4.4.2 Consecutive Memory Accesses

MC’s memory timing parameters, such as the variants of $\mathbf{tRDRD}$, specify different minimum intervals for consecutive memory accesses depending on whether they target the same bank group, different bank groups, different ranks, or different DIMMs in both Intel and AMD processors [2, 25, 42, 43]. While these timing differences encode component mapping information, they are typically small (a

²Measurements on AMD systems required careful thresholding due to limited granularity of measuring timings and various noise factors.

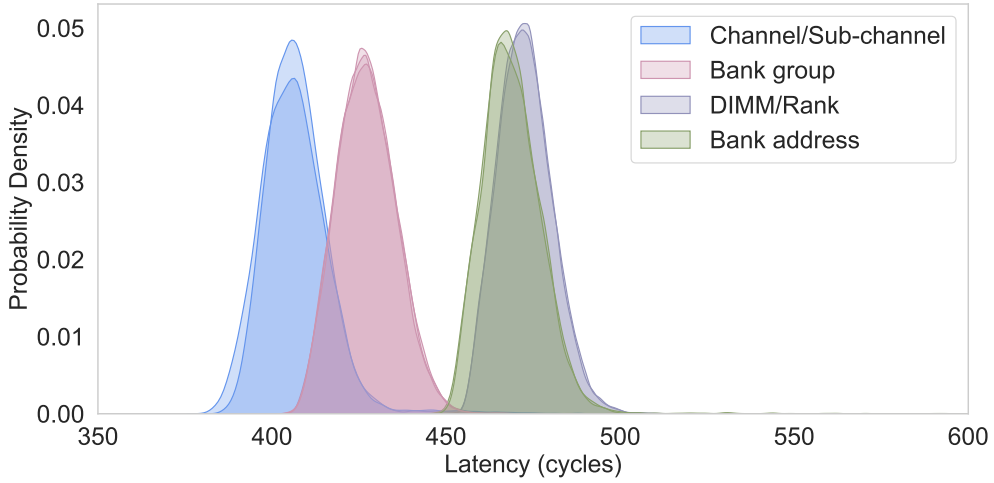


Figure 4.4: The distribution of consecutive memory access latencies across memory address pairs that differ in only one of the nine DRAM addressing functions.

few nanoseconds) and easily obscured by MC request scheduling and system noise.

To overcome this limitation, we amplify these subtle differences. Our method creates and alternately accesses two *memory streams*, each carefully constructed to ensure internal row-buffer hits. This interleaving magnifies the base timing differences dictated by the relevant tRDRD variant between the two streams, as illustrated conceptually in Figure 4.3.

Measuring the latency of these alternating stream accesses reveals distinct latency distributions (shown for an Intel Core i9-14900K in Figure 4.4). The peaks of these distributions correspond to the timing differences associated with accessing different component levels. By comparing the measured peak latencies to the system-configured tRDRD values (obtained as per Section 4.3), we can group the underlying XOR functions based on the component they likely map

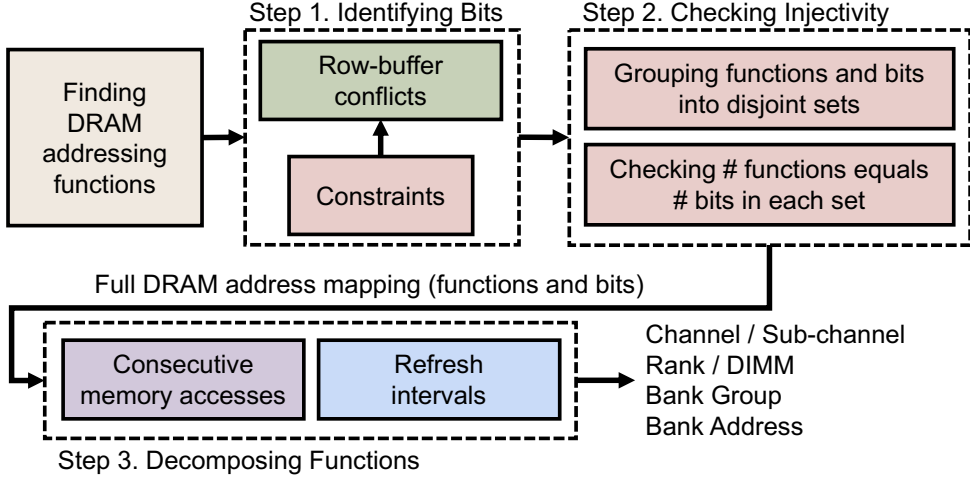


Figure 4.5: An overview of Sudoku. Sudoku i) leverages row-buffer conflicts with constraints to identify row and column bits, ii) validates the DRAM address mapping by checking injectivity, and iii) exploits refresh intervals and consecutive memory accesses to decompose DRAM address mapping.

to (*e.g.*, channel, sub-channel, bank group, DIMM/rank, and bank address). Combined, refresh interval analysis and consecutive access timing allow us to group address mapping functions by component type. However, the current method has limitations; system-configured MC timing parameters highly affect the latency of consecutive memory accesses. For example, distinguishing rank and module functions could be ambiguous if their respective trDRD timings are configured identically by the system.

4.5 Sudoku

We develop Sudoku,³ a software tool for reverse-engineering DRAM address mapping in commercial computer systems, requiring no physical access. Su-

³The tool is named Sudoku because decomposing DRAM addressing functions resembles solving a Sudoku puzzle, where each element must adhere to specific constraints.

doku extends existing tools (*e.g.*, DRAMA [104] and ZenHammer [50]) that reverse-engineer DRAM addressing functions using row-buffer conflicts. Specifically, Sudoku takes the DRAM addressing functions produced by these tools, automatically identifies row and column bits, and decomposes DRAM address mappings into component functions. As depicted in Figure 4.5, Sudoku integrates conflict-based testing [104] with timing channel analyses from Section 4.4.

4.5.1 Generating Desired Memory Addresses

To effectively isolate the timing effects of specific DRAM components (for decomposition) or row/column bits (for identification), Sudoku requires carefully crafted memory access patterns. Instead of random testing or brute-forcing, Sudoku solves the (partially) known mapping system to generate address pairs that differ only in the output of specific target functions, while keeping other function outputs identical. This targeted approach is crucial for correctly attributing observed timing variations.

Sudoku *identifies row and column bits* using row-buffer conflicts as an oracle, guided by an educated-guessing approach similar to prior work to reduce the search space [40, 146]. The key constraint is generating address pairs mapping to the same bank (*i.e.*, having identical outputs for all bank-related hash functions). Sudoku systematically tests the role of each physical address bit by generating bitmasks. It considers masks derived from the known functions and combinations involving previously unused address bits to ensure comprehensive coverage. By generating address pairs based on these masks (XORing the mask with a base address) and observing whether accesses cause row-buffer conflicts

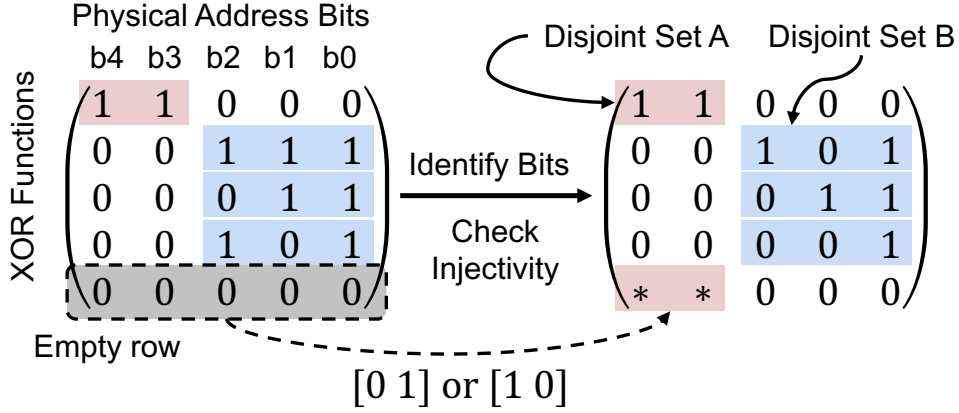


Figure 4.6: The process of verifying injectivity and finding an additional function. Disjoint set A (subsystem) doesn't satisfy the injectivity, meaning that an additional function or bit is required.

or hits, Sudoku classifies the corresponding physical address bits in the mask as contributing to row or column indexing, respectively.

4.5.2 Validating the System of Hash Functions

A valid DRAM address mapping must be *injective* (one-to-one) to ensure correctness. Sudoku verifies this property by representing the derived functions and involved bits as a linear system over $GF(2)$ and applying the rank-nullity theorem [139]. Sudoku decomposes the system into disjoint subsystems and checks if, for each subsystem, the number of linearly independent functions (rank) equals the number of involved physical address bits.

However, this conflict-based testing has inherent limitations due to the non-uniqueness of XOR-based hash systems [40, 139] (as illustrated in Figure 4.6). It may not be possible to uniquely determine every function if multiple function/bit combinations produce the same conflict behavior. To resolve such am-

biguities, Sudoku follows common assumptions (*e.g.*, assigning high-order bits to rows and low-order bits to columns [50, 104, 146]), ensuring the final mapping preserves injectivity and functional correctness, even if alternative valid representations might exist [40, 139]. Lastly, we avoid excessive function reduction (*e.g.*, reducing the system to a minimal basis) that could alter the original conflict characteristics relevant for analysis when considering both DRAM address mapping functions and row/column bits.

4.5.3 Decomposing DRAM Address Mappings

In its final step, Sudoku decomposes the validated set of DRAM addressing functions, assigning each function to its corresponding physical component (*e.g.*, channel, rank, bank group, and bank address). This decomposition leverages the distinct timing signatures identified in Section 4.4, measured using the targeted memory access patterns generated as described in Section 4.5.1. Sudoku systematically tests each relevant XOR function (or group of functions potentially related to a component) using the following timing analyses: refresh intervals and consecutive memory accesses.

Refresh Intervals: Sudoku generates pairs of addresses that differ only in the output bits produced by the target function(s). By measuring the interval between refresh-induced latency spikes when accessing these pairs (as detailed in Section 4.5.1), Sudoku identifies the functions that control the refresh group. Functions causing a “reduced” refresh interval pattern are marked as related to the refresh scope defined by the system configuration (*e.g.*, channel/sub-channel, rank/DIMM or potentially finer granularities, depending on the refresh

type employed by the system).

Consecutive Memory Accesses: To further differentiate functions, especially those not distinguished by refresh or those necessitating finer classification (like bank group vs. bank address), Sudoku employs the consecutive access timing analysis (Section 4.4.2). It generates alternating memory streams where the underlying addresses differ only based on the target function(s). The resulting latency distribution is analyzed; the location of the primary latency peak is compared against the known **trDRD** timing parameters of the system. A match allows Sudoku to associate the target function(s) with the corresponding component level (*e.g.*, mapping a function to bank group if its activation leads to latency that is consistent with **trDRD** timing for different bank groups).

By systematically applying these two timing analyses to the reverse-engineered functions using precisely controlled address generation, Sudoku labels each function with its inferred component role. This process yields the decomposed DRAM address mapping, detailing how different sets of physical address bits map to channel, sub-channel, rank/DIMM, bank group, and bank address indices.

The accuracy of this decomposition depends on the system’s behavior and configuration, as noted previously. Factors such as the MC’s refresh strategy, request scheduling policies, and whether timing parameters for different component interactions are distinct and measurable can *limit* the ability to differentiate certain functions (*e.g.*, channel vs. sub-channel, or rank vs. DIMM).

Table 4.4: Tested system configurations of Sudoku

System	Processor (microcode)	Motherboard	Memory Devices
Intel-A	Intel Core i9-12900K (0x38)	ASUS Z690-A	32 GB DDR4-3200 2R×8 UDIMM
Intel-B	Intel Core i9-12900K (0x38)	ASUS Z690-F	32 GB DDR5-4800 2R×8 UDIMM
Intel-C	Intel Core i9-14900K (0x12C)	MSI B760M	32 GB DDR5-4800 2R×8 UDIMM
AMD-A	AMD Ryzen 9 7950X (0xA601206)	ASRock X670E	32 GB DDR5-4800 2R×8 UDIMM

4.6 Results

Experimental setup: We evaluated Sudoku on the recent Intel and AMD systems detailed in Table 4.4. We measured memory access latencies in CPU cycles using the RDTSCP instruction with core dynamic frequency scaling disabled to ensure stable measurements [50,104]. For channel/sub-channel functions, which are often directly specified or easily inferred from MC-related registers (MCHBAR), we used these known values directly to constrain the reverse-engineering process, without affecting the overall mapping functionality. Lastly, we regarded recent physical probing-based DRAM address mapping results [48, 50, 88] as ground truth and verified that Sudoku generates consistent DRAM address mapping results. Table 4.5 summarizes the results of Sudoku.

4.6.1 Results: Intel Core Processor

Sudoku successfully decomposed the DRAM address mapping for the tested Intel Core 12th and 14th processors with the given DRAM addressing functions derived using a prior tool [104]. Consistent with trends observed in AMD

processors [50] and differing from some older Intel architectures [104, 146], these recent Intel platforms utilize most identified physical address bits within their mapping functions. Notably, for DDR5 configurations, we observed discontinuities in the physical address bits used for row and column indexing, influenced by multi-channel and multi-DPC (DIMM per channel) setups. Our timing analyses also confirmed the use of fine-grained all-bank refresh with DDR5, contrasting with the standard all-bank refresh observed with DDR4 on the same platform, demonstrating Sudoku’s ability to reveal such configuration details. Lastly, it is worth noting that Sudoku can identify full DRAM address mapping, thus requiring no physical access to the system.

4.6.2 Results: AMD Zen 4 Processor

On the AMD Zen 4 platform, considering the necessary PCI address offset as described in prior work [50], Sudoku produced decomposed mappings consistent with those reported by ZenHammer [50] and MCSEE [48]. The tool successfully distinguished rank/DIMM and sub-channel functions using refresh intervals and bank group and bank functions via consecutive memory accesses. The AMD processors issue separate refresh commands for each sub-channel, each DIMM, and each rank, allowing for the distinction between channel and sub-channel functions.

4.7 Discussion

4.7.1 Exploiting Memory Error Log

Recent processors provide methods for logging hardware and memory faults, such as Silent Data Corruption (SDC) [100] and machine checks [27, 58], to pro-

vide system-level diagnosis. Linux EDAC (Error Detection and Correction) is a kernel subsystem designed to detect and report hardware errors in the memory subsystem [80]. EDAC implementations vary across processor microarchitectures and manufacturers to enable precise error diagnosis. Typically, processor manufacturers develop the EDAC module to ensure its correctness. To provide DRAM component-level error analysis and repair, EDAC for server processors exposes detailed information, including the host physical address and its corresponding components (*e.g.*, MC, channel, rank, bank group, bank address, row, and column). This feature enables the reverse-engineering of DRAM addressing functions [82].

Since the EDAC source code *directly* exposes the details of DRAM addressing, EDAC can be used for the reverse-engineering of DRAM addressing functions [82]. However, EDAC primarily supports server-grade processors. Therefore, EDAC provides significantly less detailed information for desktop processors, making it insufficient for reverse-engineering full DRAM addressing functions. Moreover, since EDAC entirely relies on the processor memory controller (MCHBAR) registers [39, 42, 43], accurate knowledge of register values corresponding to various memory configurations is essential for use as in Sudoku.

4.7.2 Extending Sudoku to Other DRAMs and Hash Functions

Sudoku can be extended to other DRAM devices such as HBM (high bandwidth memory) and LPDDR (low-power DDR). Specialized memory devices have more complex internal operations and timing characteristics than stan-

dard DDR DRAMs. For example, LPDDR supports various bank modes and exhibits various DRAM timing parameters and features for low-power operations [54]. HBM provides massive parallelism by integrating more channels and stacks [53]. By leveraging Sudoku’s in-depth analysis of DRAM operations and timings, it is feasible to reverse-engineer DRAM addressing functions on these specialized memory devices.

Sudoku can be broadened to advanced memory devices integrated on-package or embedded within the system. These integrated memory devices cannot be physically probed or accessed by signal analyzers, making it infeasible to reverse-engineer. However, Sudoku can even work in these memory devices, requiring no specialized hardware [19, 75] or capability of physical accesses [50, 87, 104].

4.7.3 Use Cases of Sudoku

Sudoku can generate the desired memory access patterns based on the full DRAM addressing functions and measure the precise memory access latency. These abilities of Sudoku can be used for reverse-engineering the MC’s internal structures and other operations (*e.g.*, queue size, page policy, and scheduling policy). For example, we can automatically generate symbolic memory requests and compare their expected results and measured latencies to infer MC’s page policy and scheduling policy [37, 140].

The methodology and latency-based modeling techniques employed in Sudoku can be further extended to reverse-engineer additional DRAM-related features and other types of hash functions. Although Sudoku primarily targets

XOR-based hash functions to recover DRAM address mappings, the underlying timing channel-based approach applies to a broader range of hash functions [31, 40, 110, 144]. By collecting and validating input-output pairs through our proposed timing-based analysis, it is possible to extract microarchitectural hash functions using mathematical hash inference techniques.

Lastly, Sudoku can improve RowHammer fuzzers and attack techniques. RowHammer fuzzers [29, 49, 50] utilize DRAM addressing functions to fuzz RowHammer attack patterns efficiently. We can also improve the RowHammer fuzzers by providing more precise row and column bits and introducing an in-depth analysis of DRAM operations and their timings. Recent RowHammer attacks techniques exploits DRAM internal operations (*e.g.*, auto-refresh and in-DRAM TRR) and structures (*e.g.*, bank groups and banks) for increasing RowHammer attack success probability [23, 29, 50, 60, 87]. Sudoku’s component-level functions can be used for system-wide experiments of RowHammer attacks.

Table 4.5: Reverse-engineered DRAM addressing functions and decomposed results using Sudoku in Intel and AMD processors

Note: N/A denotes that there is no function for the corresponding component. Each value in the table represents a mask for an XOR-based hash function, where the parity of the set bits in the mask determines the output.

System	Memory Configuration	Functions				Bits	
		Channel and Sub-Channel	DIMM and Rank	Banks		Row	Column
				Bank Group	Bank Address		
Intel-A	1Ch-1DPC	N/A	0x0000088000	0x0000002A00, 0x0124044000	0x0249910000, 0x0492620000	0x07FFF80000	0x00000001FC0
	1Ch-2DPC	N/A	0x0000108000, 0x0000420000	0x00000002A00, 0x0924084000	0x0249210000, 0x0492840000	0x0FFFF80000	0x00000001FC0
	2Ch-1DPC	0x0000082600	0x0000110000	0x00000005400, 0x0248088000	0x0493220000, 0x0924040000	0x0FFFF80000	0x00000001FC0
	2Ch-2DPC	0x0000082600	0x0000210000, 0x0000840000	0x00000005400, 0x1248108000	0x0492420000, 0x0925080000	0x1FFFF00000	0x00000001FC0
Intel-B, Intel-C	1Ch-1DPC	0x00000C3200	0x0000410000	0x0000081100, 0x0222104000, 0x0442080000	0x0088820000, 0x0111040000	0x07FFF80000	0x00000000FC0
	1Ch-2DPC	0x00000C3200	0x0000810000, 0x0001040000	0x0000081100, 0x0222104000, 0x0444408000	0x0114100000, 0x088A020000	0x0FFFE80000	0x00000000FC0
	2Ch-1DPC	0x0000104200, 0x0000186400	0x0000820000	0x0000102100, 0x0444208000, 0x0888410000	0x0111040000, 0x0222208000	0x0FFFF00000	0x00000001BC0
	2Ch-2DPC	0x0000104200, 0x0000186400	0x0001020000, 0x0002080000	0x0000102100, 0x0444408000, 0x0888810000	0x0228200000, 0x1114040000	0x1FFFD00000	0x00000001BC0
AMD-A	1Ch-1DPC	0x07FFF80040	0x0000040000	0x0084200100, 0x0108400200, 0x0210801000	0x0042108000, 0x0421080400	0x07FFF80000	0x000003E080
	1Ch-2DPC	0x0FFFF00040	0x0000040000, 0x0000800000	0x0108400100, 0x0210800200, 0x0421001000	0x0084200800, 0x0842100400	0x0FFFE00000	0x000003E080
	2Ch-1DPC	0x0000000100, 0x0FFFF00040	0x0000800000	0x0108400200, 0x0210800400, 0x0421002000	0x0084201000, 0x0842100800	0x0FFFE00000	0x000007C080
	2Ch-2DPC	0x0000000100, 0x1FFFE00040	0x0000800000, 0x0000100000	0x0210800200, 0x0421000400, 0x0842002000	0x0108401000, 0x1084200800	0x1FFFE00000	0x000007C080

Chapter 5

Conclusion

5.1 Summary

Modern computer systems widely employ DRAM-based main memory systems, and various DRAM technologies and memory configurations have been introduced. As DRAM technologies evolve, the complexity of DRAM internal architectures and operations introduces new security vulnerabilities, such as DRAM read-disturbance errors and memory covert- and side-channel attacks. These increasing threats to DRAM-based memory systems necessitate a comprehensive

This chapter is based on the following works: [149, 150]. ©2023 IEEE. Reprint, with permission, from: Minbok Wi, Jaehyun Park, Seoyoung Ko, Michael Jaemin Kim, Nam Sung Kim, Eojin Lee, and Jung Ho Ahn, “SHADOW: Preventing Row Hammer in DRAM with Intra-Subarray Row Shuffling,” The 29th IEEE International Symposium on High-Performance Computer Architecture (HPCA-29), Feb 2023. Also based on: Minbok Wi, Seungmin Baek, Seonyong Park, Mattan Erez, and Jung Ho Ahn, “Sudoku: Decomposing DRAM Address Mapping into Component Functions,” arXiv preprint arXiv:2506.15918, 2025. Presented at The Fifth Workshop on DRAM Security (DRAMSec 2025) co-located with ISCA 2025.

understanding of how DRAM and the memory controllers interact. To address these needs, this dissertation analyzes and exploits the internal operations and timing behaviors of DRAM, proposing novel architectures and analysis techniques to investigate and improve the security of DRAM-based memory systems.

First, we propose SHADOW, a novel RFM-compatible in-DRAM row-shuffle-based RowHammer mitigation solution that provides superior protection capability against the non-adjacent RowHammer attacks. SHADOW adopts the newly introduced JEDEC RFM interface to trigger row-shuffle operations in DRAM without adding new DRAM commands. The row-shuffle operations of SHADOW dynamically randomize the physical adjacency between any row pair within a DRAM subarray so that it efficiently prevents non-adjacent Row Hammer attacks and attackers from generating sophisticated RH attack patterns. By populating an additional DRAM row per subarray, SHADOW efficiently tracks all of the remapping information of shuffled DRAM rows. The proposed SHADOW optimizes the access latency to the remapping information by leveraging and repurposing the DRAM internal circuitry. SHADOW identifies the row to shuffle without an additional tracking table, which would otherwise incur high area overhead. Our evaluation shows that SHADOW provides superior protection against non-adjacent Row Hammer attacks in comparison with other Row Hammer mitigation schemes and incurs little area/energy overhead.

Second, we have presented novel techniques for decomposing undocumented DRAM address mappings by exploiting previously underutilized timing channels. We first showed that DRAM refresh intervals reveal refresh group, and

second, that amplified timing variations in consecutive memory accesses expose component-level mapping information. Leveraging these insights, we developed Sudoku, a software-based tool that automatically decomposes DRAM addressing functions into their specific component roles (*e.g.*, channel, rank, bank group, and bank). We validated its effectiveness on recent Intel and AMD processors. This work provides crucial, fine-grained mapping information necessary for advanced RowHammer analysis and vulnerability assessment on modern platforms. By open-sourcing Sudoku, we hope to enable further research into DRAM security and system behavior.

In summary, this dissertation provides both architectural and analytical contributions to address growing security threats in modern DRAM-based memory systems. By deeply analyzing DRAM operations and their behaviors, our work provides new directions for building secure and practical memory systems.

5.2 Future Work

Modern memory systems are becoming increasingly complex, driven by the growing demand for high-performance, high-capacity, and energy-efficient computing. A wide range of DRAM technologies—such as LPDDR [54], GDDR [52], and HBM [53]—are now widely adopted in modern processors and systems. In parallel, heterogeneous memory systems that combine multiple types of memory (*e.g.*, DDR and non-volatile memory in [46], DDR and HBM in [126], and LPDDR and HBM in [127]) and the introduction of new memory interfaces and interconnects are actively being researched to address emerging computational needs [124].

New attack techniques and exploitation methods continue to appear, underscoring the ongoing security challenges in memory systems. For example, recent studies have demonstrated that GPUs are also susceptible to DRAM read-disturbance errors, covert- and side-channel attacks [79,95]. Adopting persistent memory has already exposed various security vulnerabilities [81,148].

Our methods for modeling and analyzing memory systems provide valuable insights and directions. The techniques that analyze DRAM operations, timing behaviors, and memory controller interactions can be extended to characterize and examine the security of the heterogeneous memory system. Additionally, these techniques can improve the security of other devices that utilize DRAM, such as GPUs and accelerators.

Bibliography

- [1] J. Ahn, S. Li, S. O, and N. P. Jouppi, “McSimA+: A Manycore Simulator with Application-level+ Simulation and Detailed Microarchitecture Modeling,” in *Proceedings of the 2013 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2013, pp. 74–85.
- [2] AMD, “BIOS and Kernel Developer’s Guide (BKDG) for AMD Family 15h Models 00h-0Fh Processors,” https://www.amd.com/content/dam/amd/en/documents/archived-tech-docs/programmer-references/42301_15h_Mod.00h-0Fh_BKDG.pdf, 2013.
- [3] S. Amer, Y. Wang, H. Kippen, T. Dang, D. Genkin, A. Kwong, A. Nelson, and A. Yerukhimovich, “PQ-Hammer: End-to-End Key Recovery Attacks on Post-Quantum Cryptography Using Rowhammer,” in *Proceedings of the 2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 3567–3582.
- [4] S. Baek, M. Wi, S. Park, H. Nam, M. J. Kim, N. Kim, and J. Ahn,

- “Marionette: A RowHammer Attack via Row Coupling,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 1, 2025, pp. 637–652.
- [5] D. Bailey, T. Harris, W. Saphir, R. Van Der Wijngaart, A. Woo, and M. Yarrow, “The NAS parallel benchmarks 2.0,” Technical Report NAS-95-020, NASA Ames Research Center, Tech. Rep., 1995.
- [6] S. Beamer, K. Asanović, and D. Patterson, “The GAP Benchmark Suite,” *arXiv preprint arXiv:1508.03619*, 2015.
- [7] T. Bennett, S. Saroiu, A. Wolman, and L. Cojocar, “Panopticon: A Complete In-DRAM Rowhammer Mitigation,” in *1st Workshop on DRAM Security (DRAMSec)*, 2021.
- [8] C. Bock, F. Brasser, D. Gens, C. Liebchen, and A.-R. Sadeghi, “RIP-RH: Preventing Rowhammer-based Inter-Process Attacks,” in *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2019, pp. 561–572.
- [9] J. Borghoff, A. Canteaut, T. Güneysu, E. B. Kavun, M. Knezevic, L. R. Knudsen, G. Leander, V. Nikov, C. Paar, C. Rechberger, R. Peter, S. S. Thomsen, and T. Yalçın, “PRINCE – A Low-Latency Block Cipher for Pervasive Computing Applications,” in *Proceedings of the 18th International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*, 2012, pp. 208–225.

- [10] F. Brasser, L. Davi, D. Gens, C. Liebchen, and A.-R. Sadeghi, “CAN’t Touch This: Software-only Mitigation against Rowhammer Attacks targeting Kernel Memory,” in *Proceedings of the 26th USENIX Security Symposium (USENIX Security 16)*, 2017, pp. 117–130.
- [11] J. Bucek, K.-D. Lange, and J. v. Kistowski, “SPEC CPU2017: Next-Generation Compute Benchmark,” in *Proceedings of the 2018 ACM/SPEC International Conference on Performance Engineering (ICPE)*, 2018, pp. 41–42.
- [12] O. Canpolat, A. G. Yağlıkçı, A. Olgun, I. E. Yuksel, Y. C. Tuğrul, K. Kanellopoulos, O. Ergin, and O. Mutlu, “BreakHammer: Enhancing RowHammer Mitigations by Carefully Throttling Suspect Threads,” in *Proceedings of the 57th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 915–934.
- [13] O. Canpolat, A. G. Yağlıkçı, G. F. Oliveira, A. Olgun, N. Bostancı, I. E. Yuksel, H. Luo, O. Ergin, and O. Mutlu, “Chronus: Understanding and Securing the Cutting-Edge Industry Solutions to DRAM Read Disturbance,” in *Proceedings of the IEEE 31st International Symposium on High-Performance Computer Architecture (HPCA)*, 2025, pp. 887–905.
- [14] O. Canpolat, A. G. Yağlıkçı, G. F. Oliveira, A. Olgun, O. Ergin, and O. Mutlu, “Understanding the Security Benefits and Overheads of Emerging Industry Solutions to DRAM Read Disturbance,” in *4th Workshop on DRAM Security (DRAMSec)*, 2024.

- [15] K. K. Chang, P. J. Nair, D. Lee, S. Ghose, M. K. Qureshi, and O. Mutlu, “Low-Cost Inter-Linked Subarrays (LISA): Enabling Fast Inter-Subarray Data Movement in DRAM,” in *Proceedings of the IEEE 22nd International Symposium on High Performance Computer Architecture (HPCA)*, 2016, pp. 568–580.
- [16] L.-P. Chang, “On Efficient Wear Leveling for Large-Scale Flash-Memory Storage Systems,” in *Proceedings of the 2007 ACM Symposium on Applied Computing (SAC)*, 2007, pp. 1126–1130.
- [17] W. Chen, Z. Zhang, X. Zhang, Q. Shen, Y. Yarom, D. Genkin, C. Yan, and Z. Wang, “HyperHammer: Breaking Free from KVM-Enforced Isolation,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, 2025, pp. 545–559.
- [18] L. Cojocar, J. Kim, M. Patel, L. Tsai, S. Saroiu, A. Wolman, and O. Mutlu, “Are We Susceptible to Rowhammer? An End-to-End Methodology for Cloud Providers,” in *Proceedings of the 2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 712–728.
- [19] L. Cojocar, K. Loughlin, S. Saroiu, B. Kasikci, and A. Wolman, “mFIT: A Bump-in-the-Wire Tool for Plug-and-Play Analysis of Rowhammer Susceptibility Factors,” *Technical Report–Microsoft Research*, 2021.
- [20] L. Cojocar, K. Razavi, C. Giuffrida, and H. Bos, “Exploiting Correcting Codes: On the Effectiveness of ECC Memory Against Rowhammer

- Attacks,” in *Proceedings of the 2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 55–71.
- [21] I. Corporation, “Intel(R) Core™ Ultra Processors for H-series and U-series Platforms CFG and MEM Registers,” <https://edc.intel.com/content/www/us/en/design/publications/14th-generation-core-processors-cfg-and-mem-registers/>, 2024, 795258-002, Accessed: April 1, 2025.
- [22] J. De Meulemeester, L. Wilke, D. Oswald, T. Eisenbarth, I. Verbauwhede, and J. Van Bulck, “BadRAM: Practical Memory Aliasing Attacks on Trusted Execution Environments,” in *Proceedings of the 2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 4117–4135.
- [23] F. de Ridder, P. Frigo, E. Vannacci, H. Bos, C. Giuffrida, and K. Razavi, “SMASH: Synchronized Many-sided Rowhammer Attacks from JavaScript,” in *Proceedings of the 30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 1001–1018. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/ridder>
- [24] F. Devaux, “The true Processing In Memory accelerator,” in *Proceedings of the 2019 IEEE Hot Chips 31 Symposium (HCS)*. IEEE Computer Society, 2019.
- [25] A. M. Devices, “BIOS and Kernel Developer’s Guide (BKDG) for AMD Family 15h Models 70h-7Fh Processors,” <https://www.amd.com/content/dam/amd/en/documents/archived-tech-docs/>

programmer-references/55072_AMD_Family_15h_Models_70h-7Fh_BKDG.pdf, 2018, 55072 Rev 3.09, Accessed: April 1, 2025.

- [26] A. Di Dio, K. Koning, H. Bos, and C. Giuffrida, “Copy-on-Flip: Hardening ECC Memory Against Rowhammer Attacks,” in *30th Annual Network and Distributed System Security Symposium (NDSS 2023)*, 2023.
- [27] X. Du, C. Li, S. Zhou, X. Liu, X. Xu, T. Wang, and S. Ge, “Fault-Aware Prediction-Guided Page Offlining for Uncorrectable Memory Error Prevention,” in *International Conference on Computer Design (ICCD)*, 2021, pp. 456–463.
- [28] S. Eyerman and L. Eeckhout, “System-Level Performance Metrics for Multiprogram Workloads,” *IEEE Micro*, vol. 28, 2008.
- [29] P. Frigo, E. Vannacc, H. Hassan, V. Van Der Veen, O. Mutlu, C. Giuffrida, H. Bos, and K. Razavi, “TRRespass: Exploiting the Many Sides of Target Row Refresh,” in *Proceedings of the 2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 747–762.
- [30] J. C. Gealow, “Impact of Processing Technology on DRAM Sense Amplifier Design,” Ph.D. dissertation, Massachusetts Institute of Technology, 1990.
- [31] L. Gerlach, S. Schwarz, N. Faröß, and M. Schwarz, “Efficient and Generic Microarchitectural Hash-Function Recovery,” in *Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 3661–3678.

- [32] D. Gruss, M. Lipp, M. Schwarz, D. Genkin, J. Juffinger, S. O’Connell, W. Schoechl, and Y. Yarom, “Another Flip in the Wall of Rowhammer Defenses,” in *Proceedings of the 2018 IEEE Symposium on Security and Privacy (SP)*, 2018, pp. 245–261.
- [33] D. Gruss, C. Maurice, and S. Mangard, “Rowhammer.js: A Remote Software-Induced Fault Attack in Javascript,” in *Proceedings of 13th Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2016, pp. 300–321.
- [34] T. R. Halfhill, “Upmem Nails RowHammer,” *Microprocessor Report, The Linley Group*, 2021.
- [35] H. Hassan, M. Patel, J. S. Kim, A. G. Yaglikci, N. Vijaykumar, N. M. Ghiasi, S. Ghose, and O. Mutlu, “CROW: A Low-Cost Substrate for Improving DRAM Performance, Energy Efficiency, and Reliability,” in *Proceedings of the ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*, 2019, pp. 129–142.
- [36] H. Hassan, Y. C. Tugrul, J. S. Kim, V. Van der Veen, K. Razavi, and O. Mutlu, “Uncovering In-DRAM RowHammer Protection Mechanisms: A New Methodology, Custom RowHammer Patterns, and Implications,” in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 1198–1213.
- [37] M. Hassan, A. M. Kaushik, and H. Patel, “Reverse-engineering Embedded Memory Controllers through Latency-based Analysis,” in *Proceedings of*

the 2015 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS), 2015, pp. 297–306.

- [38] C. Helm, S. Akiyama, and K. Taura, “Reliable Reverse Engineering of Intel DRAM Addressing Using Performance Counters,” in *Proceedings of the 2020 28th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2020, pp. 1–8.
- [39] M. Hillenbrand, “Physical Address Decoding in Intel Xeon v3/v3 CPUs: A Supplemental Datasheet,” *Karlsruhe Institute of Technology, Technical Report*, 2017.
- [40] J. Hofmann, C. Fournet, B. Köpf, and S. Volos, “Gaussian Elimination of Side-Channels: Linear Algebra for Memory Coloring,” in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024, pp. 2799–2813.
- [41] Intel, “Intel Xeon Processor 7500 Series Datasheet, Volume 2,” <https://www.intel.com/content/dam/www/public/us/en/documents/datasheets/xeon-processor-7500-series-vol-2-datasheet.pdf>, 2010.
- [42] Intel, “12th Generation Intel Core Processor Datasheet, Volume 2 of 2,” <https://www.intel.com/content/www/us/en/content-details/655259/12th-generation-intel-core-processor-datasheet-volume-2-of-2.html>, 2022, 655259-003, Accessed: November 1, 2024.

- [43] Intel, “13th Generation Intel Core Processor, Volume 2 of 2,” <https://www.intel.com/content/www/us/en/content-details/743846/13th-generation-intel-core-processors-datasheet-volume-2-of-2.html>, 2022, 743846-001, Accessed: November 1, 2024.
- [44] Intel, “Intel Memory Latency Checker v3.11,” <https://www.intel.com/content/www/us/en/developer/articles/tool/intelr-memory-latency-checker.html>, 2024.
- [45] ITRS, “ITRS Reports,” <http://www.itrs2.net/itrs-reports.html>.
- [46] J. Izraelevitz, J. Yang, L. Zhang, J. Kim, X. Liu, A. Memaripour, Y. J. Soh, Z. Wang, Y. Xu, S. R. Dulloor, J. Zhao, and S. Swanson, “Basic Performance Measurements of the Intel Optane DC Persistent Memory Module,” *arXiv preprint arXiv:1903.05714*, 2019.
- [47] B. Jacob, D. Wang, and S. Ng, *Memory Systems: Cache, DRAM, Disk*. Morgan Kaufmann, 2010.
- [48] P. Jattke, M. Marazzi, F. Solt, M. Wipfli, S. Gloor, and K. Razavi, “MC-SEE: Evaluating Advanced Rowhammer Attacks and Defenses via Automated DRAM Traffic Analysis,” in *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*, 2025.
- [49] P. Jattke, V. Van Der Veen, P. Frigo, S. Gunter, and K. Razavi, “BLACK-SMITH: Scalable Rowhammering in the Frequency Domain,” in *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 716–734.

- [50] P. Jattke, M. Wipfli, F. Solt, M. Marazzi, M. Bölskei, and K. Razavi, “ZENHAMMER: Rowhammer Attacks on AMD Zen-based Platforms,” in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1614–1633. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/jattke>
- [51] JEDEC, “DDR4 SDRAM Standard,” 2021.
- [52] JEDEC, “Graphics Double Data Rate (GDDR6) SGRAM Standard,” 2021.
- [53] JEDEC, “High Bandwidth Memory DRAM (HBM3),” 2022.
- [54] JEDEC, “Low Power Double Data Rate (LPDDR5) 5/5X,” 2023.
- [55] JEDEC, “DDR5 SDRAM,” 2024.
- [56] J. Juffinger, L. Lamster, A. Kogler, M. Eichlseder, M. Lipp, and D. Gruss, “CSI: Rowhammer–Cryptographic Security and Integrity against Rowhammer,” in *Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 1702–1718.
- [57] J. Juffinger, S. R. Neela, M. Heckel, L. Schwarz, F. Adamsky, and D. Gruss, “Presshammer: Rowhammer and Rowpress Without Physical Address Information,” in *Proceedings of 21st Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2024, pp. 460–479.

- [58] J. Jung and M. Erez, “Predicting Future-System Reliability with a Component-Level DRAM Fault Model,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023, pp. 944–956.
- [59] I. Kang, E. Lee, and J. Ahn, “CAT-TWO: Counter-Based Adaptive Tree, Time Window Optimized for DRAM Row-Hammer Prevention,” *IEEE Access*, vol. 8, 2020.
- [60] I. Kang, W. Wang, J. Kim, S. van Schaik, Y. Tobah, D. Genkin, A. Kwong, and Y. Yarom, “SledgeHammer: Amplifying Rowhammer via Bank-level Parallelism,” in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1596–1614. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/kang>
- [61] B. Keeth, R. J. Baker, B. Johnson, and F. Lin, *DRAM Circuit Design: Fundamental and High-Speed Topics*. John Wiley & Sons, 2007, vol. 13.
- [62] D.-H. Kim, P. J. Nair, and M. K. Qureshi, “Architectural Support for Mitigating Row Hammering in DRAM Memories,” *IEEE Computer Architecture Letters*, vol. 14, 2014.
- [63] D. Kim, M. Park, S. Jang, J.-Y. Song, H. Chi, G. Choi, S. Choi, J. Kim, C. Kim, K. Kim, K. Koo, S. Song, Y. Kim, D. U. Lee, J. Lee, D. Kim, K. Kwon, M. Han, B. Choi, H. Kim, S. Ku, Y. Kim, J. Kim, S. Kim, Y. Seo, S. Oh, D. Im, H. Kim, J. Choi, J. Chung, C. Lee, Y. Lee, J.-H. Cho, J. Chun, and J. Oh, “A 1.1V 1nm 6.4 Gb/s/pin 16Gb DDR5 SDRAM

with a Phase-Rotator-Based DLL, High-Speed SerDes and RX/TX Equalization Scheme,” in *Proceedings of the 2019 IEEE International Solid-State Circuits Conference-(ISSCC)*, 2019.

- [64] J. S. Kim, M. Patel, A. G. Yağlıkçı, H. Hassan, R. Azizi, L. Orosa, and O. Mutlu, “Revisiting RowHammer: An Experimental Analysis of Modern DRAM Devices and Mitigation Techniques,” in *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 638–651.
- [65] J. Kim, S. Baek, M. Wi, H. Nam, M. J. Kim, S. Lee, K. Sohn, and J. Ahn, “Per-Row Activation Counting on Real Hardware: Demystifying Performance Overheads,” *IEEE Computer Architecture Letters*, 2025.
- [66] M. Kim, J. Park, Y. Park, W. Doh, N. Kim, T. Ham, J. Lee, and J. Ahn, “Mithril: Cooperative Row Hammer Protection on Commodity DRAM Leveraging Managed Refresh,” in *Proceedings of the IEEE 28th International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 1156–1169.
- [67] M. J. Kim, M. Wi, J. Park, S. Ko, J. Choi, H. Nam, N. S. Kim, J. Ahn, and E. Lee, “How to Kill the Second Bird with One ECC: The Pursuit of Row Hammer Resilient DRAM,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2023, pp. 986–1001.

- [68] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu, “Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors,” in *Proceedings of the ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)*, 2014, pp. 361–372.
- [69] Y. Kim, V. Seshadri, D. Lee, J. Liu, and O. Mutlu, “A Case for Exploiting Subarray-Level Parallelism (SALP) in DRAM,” in *Proceedings of the 39th Annual International Symposium on Computer Architecture (ISCA)*, 2012, pp. 368–379.
- [70] A. Kogler, J. Juffinger, S. Qazi, Y. Kim, M. Lipp, N. Boichat, E. Shiu, M. Nissler, and D. Gruss, “Half-Double: Hammering From the Next Row Over,” in *Proceedings of the 31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3807–3824. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity22/presentation/kogler-half-double>
- [71] A. Kogler, D. Weber, M. Haubenwallner, M. Lipp, D. Gruss, and M. Schwarz, “Finding and Exploiting CPU Features using MSR Templating,” in *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP) IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 1474–1490.
- [72] R. K. Konoth, M. Oliverio, A. Tatar, D. Andriesse, H. Bos, C. Giuffrida, and K. Razavi, “ZebRAM: Comprehensive and Compatible Software Protection Against Rowhammer Attacks,” in *Proceedings of the 13th*

- USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 697–710.
- [73] P. Kurup and T. Abbasi, *Logic Synthesis Using Synopsys®*. Springer Science & Business Media, 2012.
- [74] A. Kwong, D. Genkin, D. Gruss, and Y. Yarom, “RAMBleed: Reading Bits in Memory Without Accessing Them,” in *Proceedings of the 2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 695–711.
- [75] D. Lee, D. Jung, I. T. Fang, C.-C. Tsai, and R. A. Popa, “An Off-Chip Attack on Hardware Enclaves via the Memory Bus,” in *Proceedings of the 29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 487–504. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity20/presentation/lee-dayeol>
- [76] D. Lee, Y. Kim, V. Seshadri, J. Liu, L. Subramanian, and O. Mutlu, “Tiered-Latency DRAM: A Low Latency and Low Cost DRAM Architecture,” in *Proceedings of the IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*, 2013, pp. 615–626.
- [77] E. Lee, I. Kang, S. Lee, G. Suh, and J. Ahn, “TWiCe: Preventing Row-hammering by Exploiting Time Window Counters,” in *Proceedings of the ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*, 2019, pp. 612–623.
- [78] S. Lee, K. Lee, M. Sung, M. Alian, C. Kim, W. Cho, R. Oh, S. O, J. Ahn, and N. Kim, “3D-XPath: High-Density Managed DRAM Architecture

- with Cost-effective Alternative Paths for Memory Transactions,” in *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2018, pp. 1–12.
- [79] C. S. Lin, J. Qu, and G. Saileshwar, “GPUHammer: Rowhammer Attacks on GPU Memories are Practical,” in *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*, 2025.
- [80] Linux, “Error Detection And Correction (EDAC) Devices,” <https://docs.kernel.org/driver-api/edac.html>, accessed: November 1, 2024.
- [81] S. Liu, S. Kanniwadi, M. Schwarzl, A. Kogler, D. Gruss, and S. Khan, “Side-Channel Attacks on Optane Persistent Memory,” in *Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 6807–6824. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/liu-sihang>
- [82] K. Loughlin, J. Rosenblum, S. Saroiu, A. Wolman, D. Skarlatos, and B. Kasikci, “Siloz: Leveraging DRAM Isolation Domains to Prevent Inter-VM Rowhammer,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 417–433.
- [83] K. Loughlin, S. Saroiu, A. Wolman, and B. Kasikci, “Stop! Hammer Time: Rethinking Our Approach to Rowhammer Mitigations,” in *Proceedings of the Workshop on Hot Topics in Operating Systems*, 2021, pp. 88–95.
- [84] H. Luo, A. Olgun, A. G. Yağlıkçı, Y. C. Tuğrul, S. Rhyner, M. B. Cavlak, J. Lindegger, M. Sadrosadati, and O. Mutlu, “RowPress: Amplifying Read

- Disturbance in Modern DRAM Chips,” in *Proceedings of the ACM/IEEE 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023, pp. 1–18.
- [85] H. Luo, T. Shahroodi, H. Hassan, M. Patel, A. G. Yağlıkçı, L. Orosa, J. Park, and O. Mutlu, “CLR-DRAM: A Low-Cost DRAM Architecture Enabling Dynamic Capacity-Latency Trade-Off,” in *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 666–679.
- [86] M. Marazzi, P. Jattke, F. Solt, and K. Razavi, “ProTRR: Principled yet Optimal In-DRAM Target Row Refresh,” in *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 735–753.
- [87] M. Marazzi and K. Razavi, “RISC-H: Rowhammer Attacks on RISC-V,” in *4th Workshop on DRAM Security (DRAMSec)*, 2024.
- [88] H. Martin and F. Adamsky, “Reverse-Engineering Bank Addressing Functions on AMD CPUs,” in *3rd Workshop on DRAM Security (DRAMSec)*, 2023.
- [89] Micron, “Calculating Memory Power for DDR4 SDRAM,” 2017.
- [90] Micron, “16Gb DDR5 SDRAM Addendum,” 2021. [Online]. Available: https://www.micron.com/-/media/client/global/documents/products/data-sheet/dram/ddr5/16gb_ddr5_sdram_diareva.pdf

- [91] M. Murugan and D. H. Du, “Rejuvenator: A Static Wear Leveling Algorithm for Flash Memory,” in *Proceedings of the 2011 IEEE 27th Symposium on Mass Storage Systems and Technologies (MSST)*, 2011.
- [92] S. Muthukrishnan, *Data Streams: Algorithms and Applications*. Now Publishers Inc., 2005.
- [93] H. Nam, S. Baek, M. Wi, M. Kim, J. Park, C. Song, N. Kim, and J. Ahn, “X-ray: Discovering DRAM Internal Structure and Error Characteristics by Issuing Memory Commands,” *IEEE Computer Architecture Letters*, vol. 22, 2023.
- [94] H. Nam, S. Baek, M. Wi, M. Kim, J. Park, C. Song, N. Kim, and J. Ahn, “DRAMScope: Uncovering DRAM Microarchitecture and Characteristics by Issuing Memory Commands,” in *Proceedings of the ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 1097–1111.
- [95] R. Nazaraliyev, Y. Zhang, S. B. Dutta, A. Marquez, K. Barker, and N. Abu-Ghazaleh, “Not so Refreshing: Attacking GPUs using RFM Rowhammer Mitigation,” in *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*, 2025.
- [96] S. O, Y. Son, N. Kim, and J. Ahn, “Row-Buffer Decoupling: A Case for Low-Latency DRAM Microarchitecture,” in *Proceedings of the ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)*, 2014, pp. 337–348.

- [97] A. Olgun, F. N. Bostancı, İ. E. Yüksel, O. Canpolat, H. Luo, G. F. Oliveira, A. G. Yağlıkçı, M. Patel, and O. Mutlu, “Variable Read Disturbance: An Experimental Analysis of Temporal Variation in DRAM Read Disturbance,” in *Proceedings of the IEEE 31st International Symposium on High-Performance Computer Architecture (HPCA)*, 2025, pp. 849–866.
- [98] A. Olgun, Y. C. Tugrul, N. Bostancı, I. E. Yuksel, H. Luo, S. Rhyner, A. G. Yaglikci, G. F. Oliveira, and O. Mutlu, “ABACuS: All-Bank Activation Counters for Scalable and Low Overhead RowHammer Mitigation,” in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1579–1596.
- [99] L. Orosa, A. G. Yaglikci, H. Luo, A. Olgun, J. Park, H. Hassan, M. Patel, J. S. Kim, and O. Mutlu, “A Deeper Look into RowHammer’s Sensitivities: Experimental Analysis of Real DRAM Chips and Implications on Future Attacks and Defenses,” in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 1182–1197.
- [100] G. Papadimitriou and D. Gizopoulos, “Silent Data Corruptions: Microarchitectural Perspectives,” *IEEE Transactions on Computers*, vol. 72, 2023.
- [101] Y. Park, W. Kwon, E. Lee, T. Ham, J. Ahn, and J. Lee, “Graphene: Strong yet Lightweight Row Hammer Protection,” in *Proceedings of the*

- 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 1–13.
- [102] M. Patel, J. S. Kim, T. Shahroodi, H. Hassan, and O. Mutlu, “Bit-Exact ECC Recovery (BEER): Determining DRAM On-Die ECC Functions by Exploiting DRAM Data Retention Characteristics,” in *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 282–297.
- [103] A. Perais and A. Sez nec, “Practical Data Value Speculation for Future High-end Processors,” in *Proceedings of the IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*, 2014, pp. 428–439.
- [104] P. Pessl, D. Gruss, C. Maurice, M. Schwarz, and S. Mangard, “DRAMA: Exploiting DRAM Addressing for Cross-CPU Attacks,” in *Proceedings of the 25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 565–581. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/pessl>
- [105] PTM, “Predictive Technology Model,” <http://ptm.asu.edu/>.
- [106] M. Qureshi, “AutoRFM: Scaling Low-Cost in-DRAM Trackers to Ultra-Low Rowhammer Thresholds,” in *Proceedings of the IEEE 31st International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.

- [107] M. Qureshi and S. Qazi, “MOAT: Securely Mitigating Rowhammer with Per-Row Activation Counters,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 1*, 2025, pp. 698–714.
- [108] M. Qureshi, S. Qazi, and A. Jaleel, “MINT: Securely Mitigating Rowhammer with a Minimalist in-DRAM Tracker,” in *Proceedings of the 57th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 899–914.
- [109] M. Qureshi, A. Rohan, G. Saileshwar, and P. Nair, “Hydra: Enabling Low-Overhead Mitigation of Row-Hammer at Ultra-Low Thresholds via Hybrid Tracking,” in *Proceedings of the ACM/IEEE 49th Annual International Symposium on Computer Architecture (ISCA)*, 2022, pp. 699–710.
- [110] M. Rainer, L. Hetterich, F. Thomas, T. Hornetz, L. Trampert, L. Gerlach, and M. Schwarz, “Rapid Reversing of Non-Linear CPU Cache Slice Functions: Unlocking Physical Address Leakage,” in *Proceedings of the 2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 3497–3515.
- [111] Rambus, “DRAM Power Model,” <http://www.rambus.com/energy>, 2010.
- [112] K. Razavi, B. Gras, E. Bosman, B. Preneel, C. Giuffrida, and H. Bos, “Flip Feng Shui: Hammering a Needle in the Software Stack,” in *Proceedings of the 25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 1–18. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/razavi>

- [113] R. Rooney and N. Koyle, “Micron® DDR5 SDRAM: New Features,” *Micron Technology Inc., Tech. Rep*, 2019.
- [114] G. Saileshwar and M. Qureshi, “MIRAGE: Mitigating Conflict-Based Cache Attacks with a Practical Fully-Associative Design,” in *Proceedings of the 30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 1379–1396. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/saileshwar>
- [115] G. Saileshwar, B. Wang, M. Qureshi, and P. Nair, “Randomized Row-Swap: Mitigating Row Hammer by Breaking Spatial Correlation between Aggressor and Victim Rows,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2022, pp. 1056–1069.
- [116] S. Saroiu, A. Wolman, and L. Cojocar, “The Price of Secrecy: How Hiding Internal DRAM Topologies Hurts Rowhammer Defenses,” in *Proceedings of the 2022 IEEE International Reliability Physics Symposium (IRPS)*, 2022, pp. 2C–3.
- [117] A. Saxena, A. Jaleel, and M. Qureshi, “ImPress: Securing DRAM Against Data-Disturbance Errors via Implicit Row-Press Mitigation,” in *Proceedings of the 57th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 935–948.
- [118] A. Saxena, S. Mathur, and M. Qureshi, “Rubix: Reducing the Overhead of Secure Rowhammer Mitigations via Randomized Line-to-Row Mapping,”

- in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, 2024, pp. 1014–1028.
- [119] A. Saxena, G. Saileshwar, J. Juffinger, A. Kogler, D. Gruss, and M. Qureshi, “PT-Guard: Integrity-Protected Page Tables to Defend Against Breakthrough Rowhammer Attacks,” in *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2023, pp. 95–108.
- [120] A. Saxena, G. Saileshwar, P. J. Nair, and M. Qureshi, “AQUA: Scalable Rowhammer Mitigation by Quarantining Aggressor Rows at Runtime,” in *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 108–123.
- [121] V. Seshadri, Y. Kim, C. Fallin, D. Lee, R. Ausavarungnirun, G. Pekhimenko, Y. Luo, O. Mutlu, P. B. Gibbons, M. A. Kozuch, and T. Mowry, “RowClone: Fast and Energy-Efficient In-DRAM Bulk Data Copy and Initialization,” in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2013, pp. 185–197.
- [122] S. M. Seyedzadeh, A. K. Jones, and R. Melhem, “Counter-Based Tree Structure for Row Hammering Mitigation in DRAM,” *IEEE Computer Architecture Letters*, vol. 16, 2017.
- [123] S. M. Seyedzadeh, A. K. Jones, and R. Melhem, “Mitigating Word-line Crosstalk Using Adaptive Trees of Counters,” in *Proceedings of the*

ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA), 2018, pp. 612–623.

- [124] D. D. Sharma, “Compute Express Link,” *CXL Consortium White Paper*, 2019.
- [125] H.-C. Shih, P.-W. Luo, J.-C. Yeh, S.-Y. Lin, D.-M. Kwai, S.-L. Lu, A. Schaefer, and C.-W. Wu, “DArT: A Component-Based DRAM Area, Power, and Timing Modeling Tool,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 33, 2014.
- [126] E. Siegmann, R. J. Harrison, D. Carlson, S. Chheda, A. Curtis, F. Coskun, R. Gonzalez, D. Wood, and N. A. Simakov, “First Impressions of the Sapphire Rapids Processor with HBM for Scientific Workloads,” *SN Computer Science*, vol. 5, 2024.
- [127] N. A. Simakov, M. D. Jones, T. R. Furlani, E. Siegmann, and R. J. Harrison, “First Impressions of the NVIDIA Grace CPU Superchip and NVIDIA Grace Hopper Superchip for Scientific Workloads,” in *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region (HPCASIA)*, 2024, pp. 36–44.
- [128] K. Sohn, T. Na, I. Song, Y. Shim, W. Bae, S. Kang, D. Lee, H. Jung, S. Hyun, H. Jeoung, K.-W. Lee, J.-S. Park, J. Lee, B. Lee, I. Jun, J. Park, J. Park, H. Choi, S. Kim, H. Chung, Y. Choi, D.-H. Jung, B. Kim, J.-H. Choi, S.-J. Jang, J.-B. Lee, and J. S. Choi, “A 1.2 V 30 nm 3.2 Gb/s/pin 4

- Gb DDR4 SDRAM with Dual-Error Detection and PVT-Tolerant Data-Fetch Scheme,” *IEEE Journal of Solid-State Circuits*, vol. 48, 2012.
- [129] Y. H. Son, S. Lee, S. O, S. Kwon, N. Kim, and J. Ahn, “CiDRA: A Cache-inspired DRAM Resilience Architecture,” in *Proceedings of the IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*, 2015, pp. 502–513.
- [130] Y. Son, S. O, Y. Ro, J. Lee, and J. Ahn, “Reducing Memory Access Latency with Asymmetric DRAM Bank Organizations,” in *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA)*, 2013, pp. 380–391.
- [131] C. Song, M. J. Kim, T. Wang, H. Ji, J. Huang, I. Jeong, J. Park, H. Nam, M. Wi, J. Ahn, and N. Kim, “TAROT: A CXL SmartNIC-Based Defense Against Multi-bit Errors by Row-Hammer Attacks,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 3*, 2024, pp. 981–998.
- [132] H. Taneja and M. Qureshi, “DREAM: Enabling Low-Overhead Rowhammer Mitigation via Directed Refresh Management,” in *Proceedings of the ACM/IEEE 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025, pp. 776–792.
- [133] A. Tatar, C. Giuffrida, H. Bos, and K. Razavi, “Defeating Software Mitigations Against Rowhammer: A Surgical Precision Hammer,” in *Pro-*

- ceedings of the 21st International Symposium on Research in Attacks, Intrusions, and Defenses (RAID)*, 2018, pp. 47–66.
- [134] S. Thoziyoor, N. Muralimanohar, J. Ahn, and N. P. Jouppi, “CACTI 5.1,” Technical Report HPL-2008-20, HP Labs, Tech. Rep., 2008.
- [135] Y. Tobah, A. Kwong, I. Kang, D. Genkin, and K. G. Shin, “SpecHammer: Combining Spectre and Rowhammer for New Speculative Attacks,” in *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 681–698.
- [136] Y. C. Tuğrul, A. G. Yağlıkçı, İ. E. Yüksel, A. Olgun, O. Canpolat, N. Bostancı, M. Sadrosadati, O. Ergin, and O. Mutlu, “Understanding RowHammer Under Reduced Refresh Latency: Experimental Analysis of Real DRAM Chips and Implications on Future Solutions,” in *Proceedings of the IEEE 31st International Symposium on High-Performance Computer Architecture (HPCA)*, 2025, pp. 867–886.
- [137] V. Van Der Veen, Y. Fratantonio, M. Lindorfer, D. Gruss, C. Maurice, G. Vigna, H. Bos, K. Razavi, and C. Giuffrida, “Drammer: Deterministic Rowhammer Attacks on Mobile Platforms,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 1675–1689.
- [138] V. Van der Veen, M. Lindorfer, Y. Fratantonio, H. Padmanabha Pillai, G. Vigna, C. Kruegel, H. Bos, and K. Razavi, “GuardION: Practical Mitigation of DMA-Based Rowhammer Attacks on ARM,” in *Proceedings of*

- 15th Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. Springer, 2018, pp. 92–113.
- [139] H. Vandierendonck and K. De Bosschere, “XOR-Based Hash Functions,” *IEEE Transactions on Computers*, vol. 54, 2005.
 - [140] P. Vila, P. Ganty, M. Guarnieri, and B. Köpf, “CacheQuery: Learning Replacement Policies from Hardware Caches,” in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2020, pp. 519–532.
 - [141] P. Vila, B. Köpf, and J. F. Morales, “Theory and Practice of Finding Eviction Sets,” in *Proceedings of the 2019 IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 39–54.
 - [142] S. Vittal, S. Qazi, P. Das, and M. Qureshi, “MoPAC: Efficiently Mitigating Rowhammer with Probabilistic Activation Counting,” in *Proceedings of the ACM/IEEE 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025, pp. 723–738.
 - [143] T. Vogelsang, “Understanding the Energy Consumption of Dynamic Random Access Memories,” in *Proceedings of the 43rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2010, pp. 363–374.
 - [144] S. Volos, C. Fournet, J. Hofmann, B. Köpf, and O. Oleksenko, “Principled Microarchitectural Isolation on Cloud CPUs,” in *Proceedings of the 2024*

ACM SIGSAC Conference on Computer and Communications Security (CCS), 2024, pp. 183–197.

- [145] C. Wang and W.-F. Wong, “Observational Wear Leveling: An Efficient Algorithm for Flash Memory Management,” in *Proceedings of the 49th ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2012, pp. 235–242.
- [146] M. Wang, Z. Zhang, Y. Cheng, and S. Nepal, “DRAMDig: A Knowledge-assisted Tool to Uncover DRAM Address Mapping,” in *Proceedings of the 57th ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6.
- [147] Y. Wang, L. Orosa, X. Peng, Y. Guo, S. Ghose, M. Patel, J. Kim, J. Luna, M. Sadrosadati, N. Ghiasi, and O. Mutlu, “FIGARO: Improving System Performance via Fine-Grained In-DRAM Data Relocation and Caching,” in *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 313–328.
- [148] Z. Wang, M. Taram, D. Moghimi, S. Swanson, D. Tullsen, and J. Zhao, “NVLeak: Off-Chip Side-Channel Attacks via Non-Volatile Memory Systems,” in *Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 6771–6788. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/wang-zixuan>
- [149] M. Wi, S. Baek, S. Park, M. Erez, and J. Ahn, “Sudoku: Decomposing DRAM Address Mapping into Component Functions,” *arXiv preprint*

arXiv:2506.15918, 2025.

- [150] M. Wi, J. Park, S. Ko, M. Kim, N. Kim, E. Lee, and J. Ahn, “SHADOW: Preventing Row Hammer in DRAM with Intra-Subarray Row Shuffling,” in *Proceedings of the IEEE 29th International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 333–346.
- [151] J. Woo, S. C. Lin, P. J. Nair, A. Jaleel, and G. Saileshwar, “QPRAC: Towards Secure and Practical PRAC-based Rowhammer Mitigation using Priority Queues,” in *Proceedings of the IEEE 31st International Symposium on High-Performance Computer Architecture (HPCA)*, 2025, pp. 1021–1037.
- [152] J. Woo and P. J. Nair, “DAPPER: A Performance-Attack-Resilient Tracker for RowHammer Defense,” in *Proceedings of the IEEE 31st International Symposium on High-Performance Computer Architecture (HPCA)*, 2025, pp. 1005–1020.
- [153] J. Woo, J. Qu, G. Saileshwar, and P. J. Nair, “When Mitigations Backfire: Timing Channel Attacks and Defense for PRAC-Based RowHammer Mitigations,” in *Proceedings of the ACM/IEEE 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025, pp. 739–756.
- [154] J. Woo, G. Saileshwar, and P. J. Nair, “Scalable and Secure Row-Swap: Efficient and Safe Row Hammer Mitigation in Memory Systems,” in *Proceedings of the IEEE 29th International Symposium on High-Performance Computer Architecture (HPCA)*, 2023, pp. 374–389.

- [155] S. C. Woo, W. Elsasser, M. Hamburg, E. Linstadt, M. R. Miller, T. Song, and J. Tringali, “RAMPART: RowHammer Mitigation and Repair for Server Memory Systems,” in *Proceedings of the International Symposium on Memory Systems*, 2023, pp. 1–15.
- [156] X.-C. Wu, T. Sherwood, F. T. Chong, and Y. Li, “Protecting Page Tables from RowHammer Attacks using Monotonic Pointers in DRAM True-Cells,” in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 645–657.
- [157] Y. Xiao, X. Zhang, Y. Zhang, and R. Teodorescu, “One Bit Flips, One Cloud Flops: Cross-VM Row Hammer Attacks and Privilege Escalation,” in *Proceedings of the 25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 19–35. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/xiao>
- [158] A. G. Yağlıkçı, J. S. Kim, F. Devaux, and O. Mutlu, “Security Analysis of the Silver Bullet Technique for RowHammer Prevention,” *arXiv preprint arXiv:2106.07084*, 2021.
- [159] A. G. Yağlıkçı, M. Patel, J. S. Kim, R. Azizi, A. Olgun, L. Orosa, H. Hassan, J. Park, K. Kanellopoulos, T. Shahroodi, S. Ghose, and O. Mutlu, “BlockHammer: Preventing Rowhammer at Low Cost by Blacklisting Rapidly-Accessed DRAM Rows,” in *Proceedings of the IEEE*

27th International Symposium on High-Performance Computer Architecture (HPCA), 2021, pp. 345–358.

- [160] A. G. Yağlıkçı, Y. C. Tuğrul, G. F. Oliveira, İ. E. Yüksel, A. Olgun, H. Luo, and O. Mutlu, “Spatial Variation-Aware Read Disturbance Defenses: Experimental Analysis of Real DRAM Chips and Implications on Future Solutions,” in *Proceedings of the IEEE 30th International Symposium on High-Performance Computer Architecture (HPCA)*, 2024, pp. 560–577.
- [161] F. Yao, A. S. Rakin, and D. Fan, “DeepHammer: Depleting the Intelligence of Deep Neural Networks through Targeted Chain of Bit Flips,” in *Proceedings of the 29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 1463–1480. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity20/presentation/yao>
- [162] S. F. Yitbarek, M. T. Aga, R. Das, and T. Austin, “Cold Boot Attacks are Still Hot: Security Analysis of Memory Scramblers in Modern Processors,” in *Proceedings of the IEEE 23rd International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 313–324.
- [163] J. M. You and J.-S. Yang, “MRLoc: Mitigating Row-hammering Based on Memory Locality,” in *Proceedings of the 56th ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2019, pp. 1–6.
- [164] T. Zhang, K. Chen, C. Xu, G. Sun, T. Wang, and Y. Xie, “Half-DRAM: a High-bandwidth and Low-power DRAM Architecture from the Re-

- thinking of Fine-grained Activation,” in *Proceedings of the ACM/IEEE 41st Annual International Symposium on Computer Architecture (ISCA)*, 2014, pp. 349–360.
- [165] Z. Zhang, Y. Cheng, D. Liu, S. Nepal, Z. Wang, and Y. Yarom, “PThammer: Cross-User-Kernel-Boundary Rowhammer through Implicit Accesses,” in *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 28–41.
- [166] Z. Zhang, Y. Cheng, M. Wang, W. He, W. Wang, S. Nepal, Y. Gao, K. Li, Z. Wang, and C. Wu, “SoftTRR: Protect Page Tables against Rowhammer Attacks using Software-only Target Row Refresh,” in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 2022, pp. 399–414.

국문초록

DRAM (Dynamic Random-Access Memory)은 현대 컴퓨터 시스템에서 가장 일반적으로 사용되는 주기억장치이며, 전체적인 시스템의 성능, 신뢰성, 그리고 보안에 직접적으로 영향을 미친다. 그러나 DRAM 내부 구조가 점점 더 복잡해지고, DRAM 장치는 메모리 컨트롤러에 의해 동작이 통제되는 주-종속적 인터페이스를 따르기 때문에, 시스템 차원에서 DRAM의 성능 병목, 신뢰성 문제, 그리고 보안 취약점을 체계적으로 분석하는 데 어려움이 있다. 따라서 DRAM 기반 메모리 시스템의 동작을 정확하게 이해하고 분석하는 것은 고신뢰성, 고성능 시스템 설계를 위해 필수적이다.

본 논문은 DRAM 기반 메모리 시스템의 보안성과 신뢰성 향상을 목표로, DRAM의 내부 동작을 체계적으로 분석하고 이를 활용하는 새로운 방법론을 제안한다. 먼저, DRAM 내부 명령어를 활용한 효율적인 데이터 복사 방식을 통해, DRAM의 대표적인 보안 취약점인 로우해머(RowHammer) 공격에 효과적으로 대응할 수 있는 하드웨어 구조를 제안한다. 또한, DRAM 및 메모리 컨트롤러의 동작 특성을 기반으로 시스템 수준의 메모리 접근 지연시간을 분석하고, 이를 활용하여 DRAM 내부 동작을 추론하는 마이크로벤치마크 기반 분석 기법을 제안한다. 이 분석을 바탕으로, 상용 DRAM과 메모리 컨트롤러의 구조와 동작 특성을 밝히는 역공학 기법을 제안한다.

첫째로, DRAM의 대표적인 취약점인 로우해머 공격에 대해 효율적인 방어를 수행하는 하드웨어 기반 DRAM 내부 로우해머 방지 기법 SHADOW를 제안한다. SHADOW는 로우-셔플링(row-shuffling)을 활용하여, DRAM 서브어레이 내

로우 간 물리적 인접성을 동적으로 무작위함으로써, 기존 로우해머 공격과 새롭게 제안된 비인접 로우해머 공격 및 정교한 로우해머 공격 패턴에 대해서 효율적으로 방어한다. SHADOW는 JEDEC 표준의 RFM 인터페이스를 기반으로 DRAM 내부에서 로우-셔플링이 수행되도록 하며, DRAM 내부 명령어 기반의 로우-복사 (row-copy)를 통해 효율적으로 DRAM 내부 로우-셔플링을 구현하여, 데이터 이동에 따른 성능 및 에너지 오버헤드를 최소화한다. 제안하는 구조는 서브레이당 하나의 추가 DRAM 로우를 사용하여 DRAM 로우의 재배치 정보를 효과적으로 관리하고, DRAM 내부 회로를 재활용하여 접근 시간 지연을 최소화한다.

둘째로, DRAM 기반 메모리 시스템의 내부 구조 및 지연시간 특성 분석을 통해, DRAM 주소 매핑을 구성요소 단위의 매핑 함수로 분해하는 새로운 분석 기법을 제안한다. DRAM 주소 매핑을 채널, 랭크, 뱅크 그룹, 뱅크, 로우, 컬럼의 구성요소별 함수로 분해하는 것은 정확한 메모리 동작 이해 및 로우해머 분석을 위해 필수적이다. 그러나 기존 역공학 기법으로는 물리적 접근 없이 이를 수행하는 데 한계가 있다. 이에 본 논문은 DRAM의 리프레시 간격과 연속적인 메모리 접근 패턴을 활용하여 각 구성요소 별 주소 매핑 함수를 추론할 수 있는 지연시간 기반 분석 기법을 제안한다. 이 방법론을 바탕으로, 채널, 랭크, 뱅크 그룹, 뱅크, 로우, 컬럼 비트까지 정확히 식별할 수 있는 소프트웨어 기반의 주소 매핑 역공학 도구 Sudoku를 구현한다. Sudoku의 유효성을 검증하기 위해 최신 Intel 및 AMD 프로세서를 대상으로 실험을 수행하였으며, 복잡한 DRAM 주소 매핑을 구성요소 단위로 성공적으로 분해할 수 있음을 확인하였다.

본 논문에서 제시한 DRAM 내부 명령어 기반 기법과 시스템 수준에서의 DRAM 지연시간 분석 방법은, 현대 DRAM 기반 메모리 시스템의 성능, 보안성, 그리고 신뢰성 향상에 기여할 수 있다. 특히 SHADOW에서 활용한 DRAM을 활용한 데이터 처리 방식인 DRAM 내부 로우 셔플링 기법은, DRAM의 구조적

특성을 활용하여 데이터 무결성과 보안성을 강화할 수 있음을 보여주는 대표적인 사례이다. 또한 Sudoku의 심층적인 DRAM 및 메모리 컨트롤러의 동작 분석 및 주소 매핑 역공학 기법은, 점차 복잡해지는 DRAM 기반 메모리 시스템에 대해 보다 정밀한 성능 및 보안 분석의 기반을 제공한다. 마지막으로, 본 논문에서 개발한 소프트웨어 도구와 실험 결과는 향후 DRAM 구조 분석, 성능 최적화, 그리고 보안 및 신뢰성 강화 연구에 활용 가능한 실용적인 기반을 제공할 것으로 기대된다.

주요어: 메모리 시스템, DRAM, 메모리 컨트롤러, 로우해머, 타이밍 채널 분석, 마이크로벤치마킹, DRAM 주소 매핑, 역공학

학번: 2018-24285